RAZORPAY AI BUILDATHON 2026 / TRACK 3 / SOLO

Hinglish Revenue *Recovery Engine*

A **bounded, measured, multilingual** payment-failure recovery pipeline built on Razorpay test mode. Combines a calibrated P(recovery) classifier, decline-code-aware routing, AST-enforced policy gates, and a 7-language communication generator — with honest metrics on a held-out batch.

Track 3 · AI Revenue Recovery | Measured P(recovery) | Multilingual · 7 langs | AST-enforced gates | Closed-loop webhook | Bootstrap 95% CI

POSITIONING SENTENCE

The only Track 3 submission with (a) real Razorpay test-mode closed-loop recovery, (b) P(recovery) measured — not heuristic — on a held-out batch with precision/recall and a retry-everything baseline, (c) decline-code-aware routing, and (d) multilingual customer comms — under an AST-enforced policy gate with a replayable audit ledger.

Solo Builder Strategy Document

BUILD STRATEGY · ARCHITECTURE · PITCH SCRIPT

VERSION

September 2026

Table of Contents

Table of Contents 1

Executive Summary 5

Three Target KPIs . . . 5

Competition Landscape & Track Selection 7

Track Decision Matrix . . . 7

Competitor Teardown — 14 Repos Analyzed 9

Track 3 Direct Competitors . . . 9

Out-of-Track Leaders to Emulate . . . 9

Strategic Positioning — Four Pillars of Differentiation 11

Pillar 1: Real Razorpay Test-Mode Closed-Loop Recovery . . . 11

Pillar 2: P(Recovery) Measured on a Held-Out Batch . . . 11

Pillar 3: Decline-Code-Aware Routing . . . 12

Pillar 4: AST-Enforced Policy Gate with Multilingual Comms . . . 12

Why AI is Genuinely Needed (AI Judgment Criterion) . . . 13

System Architecture Overview 14

Why a Pipeline, Not an Agent . . . 14

Component Interaction Matrix . . . 15

Synthetic Dataset — Schema & Generation Strategy 16

Schema Definition . . . 16

Distribution Rules per Error Code . . . 17

Three Representative JSON Records . . . 17

ML Model — Calibrated P(Recovery) Classifier 20

Why a Calibrated Gradient-Boosted Model . . . 20

Feature Engineering . . . 20

Training Pipeline (Code Snippet) . . . 21

When the Model Fails . . . 22

Decline-Code-Aware Routing Engine 23

Error-Code → Channel Mapping . . . 23

Webhook Handler — Closed-Loop Implementation . . . 24

Bounded Recovery Workflow — Stopping Rules & Audit Trail 26

The Five Policy Gates . . . 26

Policy Gate Implementation (Code Snippet) . . . 26

AST-Enforced Invariant . . . 28

Multilingual Communication Generator — Hinglish & Beyond 30

Architecture: Skeleton + Slot-Fill . . . 30

Hinglish Sample Exhibit Boxes . . . 30

Audit Property: Reproducibility . . . 31

Failure Modes & Graceful Fallbacks 33

Failure Mode Catalog . . . 33

Detailed Walkthrough: Failure Mode 1 (LLM Hallucinates UPI Handle) . . . 33

Measurement Dashboard & Honest Metrics 36

Six Tracked Metrics . . . 36

Three Baselines . . . 37

Honest Exception List . . . 37

Failure-Decision Tree (Visual) 39

AST-Enforced: No LLM in the Tree . . . 39

Per-Payment Audit Ledger Schema . . . 40

9-Day Build Plan (Visual Gantt) 41

Day-by-Day Breakdown . . . 41

Risk/Decision Matrix . . . 42

5-Minute Pitch Video Script Template 43

Segment 1: Hook (0:00 — 0:30, 30 seconds) . . . 43

Segment 2: Problem Context (0:30 — 1:30, 60 seconds) . . . 43

Segment 3: Architecture & AI Justification (1:30 — 2:30, 60 seconds) . . . 44

Segment 4: Live Demo (2:30 — 4:00, 90 seconds) . . . 44

Segment 5: Metrics & Honest Exceptions (4:00 — 5:00, 60 seconds) . . . 44

Segment 6: Close (5:00 — 5:30, 30 seconds) . . . 45

Three Sound-Bite Sentences to Memorize . . . 45

README Template (Copy-Paste Ready) 46

Submission Checklist 54

Tier 1: MUST-HAVE (Disqualifying if Missing) . . . 54

Tier 2: SHOULD-HAVE (Differentiators) . . . 54

Tier 3: NICE-TO-HAVE (Polish) . . . 55

Final 60-Minute Pre-Deadline Checklist . . . 55

Risk/Decision Matrix — When to Pivot 57

Day 2 Kill Gate — Classifier AUC . . . 57

Day 3 Kill Gate — Webhook Closed Loop . . . 57

Day 4 Kill Gate — Policy Gate + AST Invariant . . . 58

Day 5 Kill Gate — Multilingual Comms . . . 58

Day 6 Kill Gate — Dashboard + Metrics 58

Track 2 Fallback Plan 59

Closing — Why This Wins **60**

Four Pillars Recap 60

What to Do Today (Next 24 Hours) 60

One-Sentence Elevator Pitch (Memorize) 61

Executive Summary

This document is the complete build strategy for a solo submission to the Razorpay AI Buildathon 2026, Track 3 (AI Revenue Recovery). The submission is a payment-failure recovery pipeline that listens to Razorpay `payment.failed` webhooks in test mode, classifies each failure with a calibrated machine-learning model, applies five deterministic policy gates, renders multilingual customer-facing recovery messages in seven Indian languages, executes a bounded retry sequence, and measures the money recovered against three explicit baselines on a held-out batch.

The strategy is built around one observation from competitor research: of the 14 public GitHub repositories analysed, five are actual Track 3 competitors, and none of them simultaneously has (a) real Razorpay test-mode closed-loop recovery, (b) P(recovery) measured on a held-out batch with precision/recall and baselines, (c) decline-code-aware routing, and (d) multilingual customer comms. The single combination of these four properties is the empty cell in the competitive landscape, and this submission is designed to occupy it completely.

> **POSITIONING SENTENCE (MEMORIZE THIS)**
> The only Track 3 submission with (a) real Razorpay test-mode closed-loop recovery, (b) P(recovery) measured — not heuristic — on a held-out batch with precision/recall and a retry-everything baseline, (c) Razorpay-decline-code-aware routing, and (d) multilingual customer comms — all under an AST-enforced policy gate with a replayable per-payment audit ledger.

The four differentiators above are not independent marketing claims; they reinforce each other. Measured P(recovery) is only credible when the policy gates are bounded and audited, because otherwise the "recovered" counter is easy to game. Multilingual comms only matter when the recovery loop is closed, because otherwise the customer receives a message in their preferred language and then nothing happens. Decline-code-aware routing only improves recovery when paired with a calibrated model, because the model assigns the per-code recovery probability that the routing decision depends on. The architecture is one coherent thesis, not four bolted-on features.

Three Target KPIs

RECOVERY LIFT VS BASELINE	LANGUAGES COVERED	AST-ENFORCED INVARIANTS
+163%	**7**	**6**
vs retry-everything	en, hi, ta, te, kn, mr, bn	build fails if violated

These three KPIs anchor the pitch video, the README, and the live demo. The first is the headline number — the additional money recovered on the held-out batch versus the strongest baseline. The second is the count of customer-preferred languages supported end-to-end (locale detection, message generation, voice IVR script). The third is the number of invariants enforced

by a static analysis pass on the codebase, which makes the bounded-explainability claim machine-verifiable rather than prose-only.

The remainder of this document is structured as a build plan, not a research paper. Sections 1–3 cover competitive landscape and strategic positioning. Sections 4–9 are the technical architecture, dataset, model, routing engine, policy gate, and communication generator. Sections 10–12 cover failure modes, metrics, and the decision tree. Sections 13–17 are the build plan, pitch script, README template, submission checklist, and risk matrix. The document is dense by design — it is meant to be read once, used as a reference during the nine-day build, and then condensed into the five-minute pitch video and the README.

Competition Landscape & Track Selection

The Razorpay AI Buildathon 2026 offers five tracks: Agentic Commerce (Track 1), AI Risk Manager (Track 2), AI Revenue Recovery (Track 3), AI Finance Controller (Track 4), and an Open Track (Track 5). All tracks are judged on four universal criteria that are hidden inside the track descriptions: AI Judgment (was AI genuinely needed?), Failure Recovery (real runtime failure modes with graceful fallbacks), Measured Honesty (explicit metrics with exception lists), and Bounded Explainability (audit trail and stopping rules).

Selection in Round 2 is based purely on the submission — the public GitHub repository, a five-minute pitch video, and architecture documentation that explicitly includes "what broke and how you recovered it". There is no resume screen and no group discussion. This means the strategy is to maximise the quality of the artefacts, not the credentials of the builder.

Track Decision Matrix

Two tracks were considered seriously: Track 3 (Revenue Recovery) and Track 2 (Risk Manager). Track 3 was chosen as the primary track. The matrix below shows the reasoning, calibrated to a nine-day solo build by a participant with prior fraud-detection and multilingual NLP infrastructure.

Factor	Track 3: Revenue Recovery	Track 2: Risk Manager
Existing code reuse	Medium — multilingual infra reusable	High — fraud repo + calibration
Differentiation among applicants	High — multilingual is rare	Low-Medium — common ML task
Razorpay alignment	High — Agent Studio has recovery agents	Medium — Thirdwatch already exists
Build risk on 9-day timeline	Medium — more components	Low — one loss class
"Measured" metric achievability	Medium — money recovered needs setup	High — precision/recall clean
`Ceiling / wow factor`	High — Hinglish voice is memorable	Medium — evaluated ML
Competitor density (from research)	5 serious repos, all beatable	2 gold-standard repos to beat
Decision	CHOSEN	Fallback if Day 2 fails

The decisive factors were competitor density and differentiation. Track 2 has two submissions (yvssshubam and prahari-ai) that have already set a high bar for evaluation discipline: both have measured holdouts, bootstrap confidence intervals, baseline comparisons, and honest cost-sweep results that publish the regime where their model loses. Beating them on Track 2 in nine days would require matching their evaluation rigor and then beating it — possible, but high-effort.

Track 3, by contrast, has five serious competitors, none of whom has measured recovery rates. They use hardcoded heuristics (madhav9757: 0.85 / 0.60 / 0.40), hand-written logistic functions (aniket-gawande), or simulated outcomes against a stub (PRANEETHWARANK). The opportunity is to bring Track 2's evaluation discipline to Track 3, where nobody has done it yet. That is the strategic move: import the rigor of the strongest Track 2 submissions into the empty cell of Track 3, add multilingual comms (zero competitors have it), and ship under an AST-enforced policy gate (only prahari-ai has invariants, and only on Track 2).

> **WHY AI IS THE RIGHT TOOL HERE, NOT FORCED**
> AI is used in two places only: (1) a calibrated gradient-boosted classifier predicting P(recovery | features) — beats heuristic lookup tables because Razorpay's error_code taxonomy is too rich (50+ codes) for hand-coded rules; (2) an LLM filling slot variables inside deterministic multilingual message skeletons — beats template-only approaches because Hindi/Tamil/Telugu etc. have inflection that templates cannot capture. Every other component (webhook verification, policy gates, retry sequencing, audit logging) is deterministic Python. The AI Judgment criterion is satisfied because the AI is in the parts where it strictly outperforms rules.

Competitor Teardown — 14 Repos Analyzed

Fourteen public repositories were analysed in detail. Source: GitHub API + raw README fetch + code clone for the eight repos with the most substance. The full per-repo analysis is summarised below in two tables: the five real Track 3 competitors first, then the two out-of-track leaders whose evaluation discipline should be emulated.

Track 3 Direct Competitors

Repo	Razorpay	Measured Metrics	Multilingual	Audit Trail	Weakness to Exploit
madhav9757	Real (closed loop)	Runtime only; heuristic probs	No	In-memory metrics	Hardcoded 0.85/0.60/0.40
aniket-gawande	Real (webhook)	Simulated; hand-written logistic	No	Prisma + red-team suite	No root README; heuristic
VISHAL55UIET	Real (order+verify)	None; uncalibrated LLM conf.	No	Saga events	1,829-line README, no holdout
PRANEETHWARANK	STUB (faked)	100% precision (suspicious)	No	SQLite + JSONL	Fakes Razorpay; synthetic 100%
joshikarthikeyan4	None	None; hardcoded % per code	No	None	No Razorpay, Streamlit only

The pattern across all five is consistent. Every competitor either uses heuristic recovery probabilities or simulates the recovery outcome. Nobody has trained P(recovery | features) on a labeled holdout and reported precision/recall against baselines. The competitor with the best-looking numbers (PRANEETHWARANK: 100% precision, 0% FPR) achieves them by simulating the recovery outcome from the same generator that produced the labels — the 100% is a property of the generator, not the model. The competitor with the strongest engineering (VISHAL55UIET: Saga, idempotency, Resilience4j) reports raw LLM confidence (0.88) as if it were a calibrated probability, which it is not.

Two competitors have genuine Razorpay test-mode integration with a closed loop (madhav9757) or order+verify (VISHAL55UIET), but neither routes on Razorpay's actual error_code taxonomy with calibrated per-code lift. Three competitors have no Razorpay integration at all, or fake it with a stub that returns the ground-truth label as the "recovery outcome".

Out-of-Track Leaders to Emulate

Two Track 2 submissions set the bar for evaluation discipline across the entire field. They are not direct competitors (they build chargeback triage, not payment-failure recovery), but their patterns should be imported into the Track 3 submission.

Repo	What They Got Right	How to Import It
yvssshubam (Track 2)	Held-out 800 disputes; bootstrap 95% CI ₹200K-₹265K; honest negative result that model ≈ heuristic on net rupees; 9 documented-and-corrected errors	Bring this level of evaluation discipline to Track 3 — nobody has it. Holdout set, two baselines, bootstrap CI, ablation vs hand-written heuristic
`kanikakataria75 / prahari-ai (Track 2)`	6 AST-enforced invariants; 905 tests; full degradation table; measured calibrator selection (raw booster beats isotonic, shipped honestly); publishes cost regime where it loses	AST-enforced invariants + degradation table — cheap to add, reads as engineering maturity. Adopt the "publish your losing regime" honesty

The key insight: the two best submissions in the field are both on Track 2, where evaluation discipline is the bar. Track 3 has no such submission. The strategic move is to bring Track 2's evaluation rigor into Track 3, where it will be the only submission with measured recovery metrics, and pair it with the multilingual capability that nobody has.

COMMON GAPS ACROSS ALL 14 REPOS

(1) No measured P(recovery) — every Track 3 competitor uses heuristics or simulated outcomes. (2) No baseline comparison — only Track 2 leaders compare against retry-everything / retry-nothing / oracle. (3) No multilingual customer comms anywhere — zero repos support Hindi/Tamil/Telugu etc. (4) No decline-code-aware routing grounded in Razorpay's actual error_code taxonomy. (5) No durable, queryable audit ledger in the Track 3 field that reconstructs a full decision → attempt → outcome → cost trace per payment.

Strategic Positioning — Four Pillars of Differentiation

The strategy is built on four pillars, each addressing a gap in the competitor field. Each pillar is independently defensible — even if one is weakened during the build, the other three still differentiate the submission. Together, they form a position that no competitor currently occupies.

Pillar 1: Real Razorpay Test-Mode Closed-Loop Recovery

The closed loop is: `payment.failed` webhook → policy gate → create Razorpay Payment Link → customer pays → `payment_link.paid` webhook → mark recovered. This is the only honest way to claim "money recovered" — the recovered counter increments only when Razorpay confirms the payment. Of the five Track 3 competitors, only madhav9757 has this closed loop, and even there it routes on heuristic probabilities. The other four either have no Razorpay integration, a one-way webhook, or a stub that fakes the recovery outcome.

Implementation detail: the Razorpay Node SDK or Python SDK is invoked in test mode, which simulates the full payment flow without moving real money. Test-mode payment links are real Razorpay objects with real IDs; the only difference from production is that the customer pays with a test card or test UPI handle. This means the demo can show a real Razorpay dashboard with real payment-link objects, real webhook deliveries, and real HMAC signature verification — not a mock.

Pillar 2: P(Recovery) Measured on a Held-Out Batch

The classifier predicts the probability that a given payment failure is recoverable, conditioned on features (error_code, customer_locale, prior_failure_count, payment_method, amount_bucket, time_of_day_ist, merchant_segment). It is a calibrated gradient-boosted model — isotonic regression on top of XGBoost or LightGBM. The model is trained on 70% of the synthetic dataset and evaluated on a 30% holdout that the model never sees.

Three baselines are computed on the same holdout: retry-nothing (recovers ₹0), retry-everything (recovers some money but at high false-positive cost), and rule-based (recovers money using a hand-coded lookup table of per-code recovery rates, à la madhav9757). The model's net recovered rupees, precision, recall, Brier score, and ECE are reported with bootstrap 95% confidence intervals from 2000 resamples. This is the evaluation discipline that yvssshubam and prahari-ai brought to Track 2, here applied to Track 3 where nobody has it.

> **WHY A CALIBRATED MODEL, NOT AN LLM, FOR P(RECOVERY)**

> An LLM cannot give calibrated probabilities — its "confidence" is a likelihood score, not a frequentist probability, and there is no mechanism to verify it against ground truth. A gradient-boosted model with isotonic calibration, evaluated on a held-out batch with Brier score and ECE, gives a probability that is empirically correct on average. This matters because the policy gate (Pillar 4) uses p_recovery in its EVR calculation: p × amount − cost > 0. If p is uncalibrated, the gate is fiction. This is the exact failure mode of VISHAL55UIET, which reports raw LLM confidence (0.88) as if it were a probability.

Pillar 3: Decline-Code-Aware Routing

Razorpay's payment failure payloads contain a structured `error_code` field from a published taxonomy (insufficient_balance, network_error, authentication_failed, mandate_invalid, card_expired, suspected_fraud, etc.). Each error code has a different recoverability profile: insufficient_balance is highly recoverable (customer has funds elsewhere), suspected_fraud is not recoverable (retrying fraud is disqualifying), mandate_invalid requires a customer action (re-mandate), authentication_failed requires re-authentication.

The routing engine maps each error_code to a recovery channel: retry (same method, technical failure), payment_link_update (re-issue with different method), voice_call (mandate issues, customer needs to talk), reauth (authentication issues), or block (fraud, halt immediately). The mapping is deterministic and reviewable — it is not a model output. The model's job is to provide p_recovery; the routing engine's job is to choose the channel. This separation is what makes the system auditable: every routing decision can be traced to (error_code → channel mapping) + (p_recovery → gate decision).

Stripe's Smart Retries is the reference competitor here. It is a black-box ML-based retry timing system — it does not coordinate with dunning, does not respect the customer's preferred language, and does not expose why a retry was scheduled at a particular time. Purpose-built dunning tools (Butterfly, Recurly, Stripe Billing) add roughly 20% recovery on top of black-box retry, by coordinating timing with customer communication. The opportunity is decline-code-aware routing: instead of treating all failures the same and only varying the timing, vary the channel based on the specific failure reason.

Pillar 4: AST-Enforced Policy Gate with Multilingual Comms

The policy gate is a sequence of five deterministic checks applied to every payment failure: (1) is the failure recoverable per the model? (2) is the velocity within limits (⩽ 3 failures per customer per 60 seconds)? (3) is the current time outside the TRAI DND window (21:00–09:00 IST)? (4) is the expected value of recovery positive (EVR = p × amount − cost > 0)? (5) is the retry count below the cap (< 3)? If any gate fails, the recovery attempt is halted and the decision is logged to the audit ledger.

The AST-enforced invariant is that no LLM call may modify the decision tree — the LLM only executes inside it (for message rendering). This is enforced by a Python AST pass that walks the codebase and fails the build if any function outside a whitelist calls the LLM client. This is the pattern from prahari-ai (Track 2): the LLM is bounded to a specific operation, and a static check makes the bound machine-verifiable.

Multilingual comms is the fourth differentiator because it is literally zero across the field. Razorpay's Agent Studio has Subscription Recovery and Abandoned Cart Conversion agents, but neither supports Hindi, Tamil, Telugu, Kannada, Marathi, or Bengali. For an Indian payment-recovery product where the end customer often prefers a regional language, this is both a real product wedge and a clean competitive differentiator. The LLM fills slots (amount, UPI handle, merchant name, link) inside a deterministic skeleton — it never invents the amount or the UPI handle, only renders the surrounding sentence in the customer's preferred language.

Why AI is Genuinely Needed (AI Judgment Criterion)

The AI Judgment criterion asks whether AI was used because it genuinely fit the problem, or just forced in. The submission uses AI in exactly two places: (1) the calibrated classifier for P(recovery), and (2) the LLM for multilingual message generation. In both places, the AI strictly outperforms the simpler alternative.

For the classifier: a rule-based lookup table of per-code recovery rates is the simpler alternative, and it is what every Track 3 competitor uses. But Razorpay's error_code taxonomy has 50+ codes, and recovery probability depends on contextual features (customer_locale, prior_failure_count, time_of_day) that a per-code lookup cannot capture without exploding into a combinatorial table. A gradient-boosted model handles the interactions natively, and isotonic calibration makes its output a true probability usable in the EVR gate.

For the LLM: a deterministic template per language is the simpler alternative, and it is what a naive implementation would do. But Hindi, Tamil, Telugu, and Bengali have inflection (verb conjugation, noun declension, honorific levels) that depends on the customer's segment, the amount, and the merchant name. A template approach either drops the inflection (sounds robotic) or hard-codes every variant (explodes combinatorially). The LLM fills slots inside a deterministic skeleton — the skeleton fixes the structure and the slot values (so the LLM cannot invent amounts or UPI handles), and the LLM only handles the inflection. This is the minimal use of AI that solves the problem.

System Architecture Overview

The system is a six-component pipeline, not an "agent". Every component has explicit inputs, outputs, and stopping rules. The LLM is bounded to a specific operation (multilingual message generation) inside a deterministic skeleton; it never decides whether to recover, when to retry, or which channel to use. The architecture diagram below shows the data flow; the table that follows summarises each component's contract.

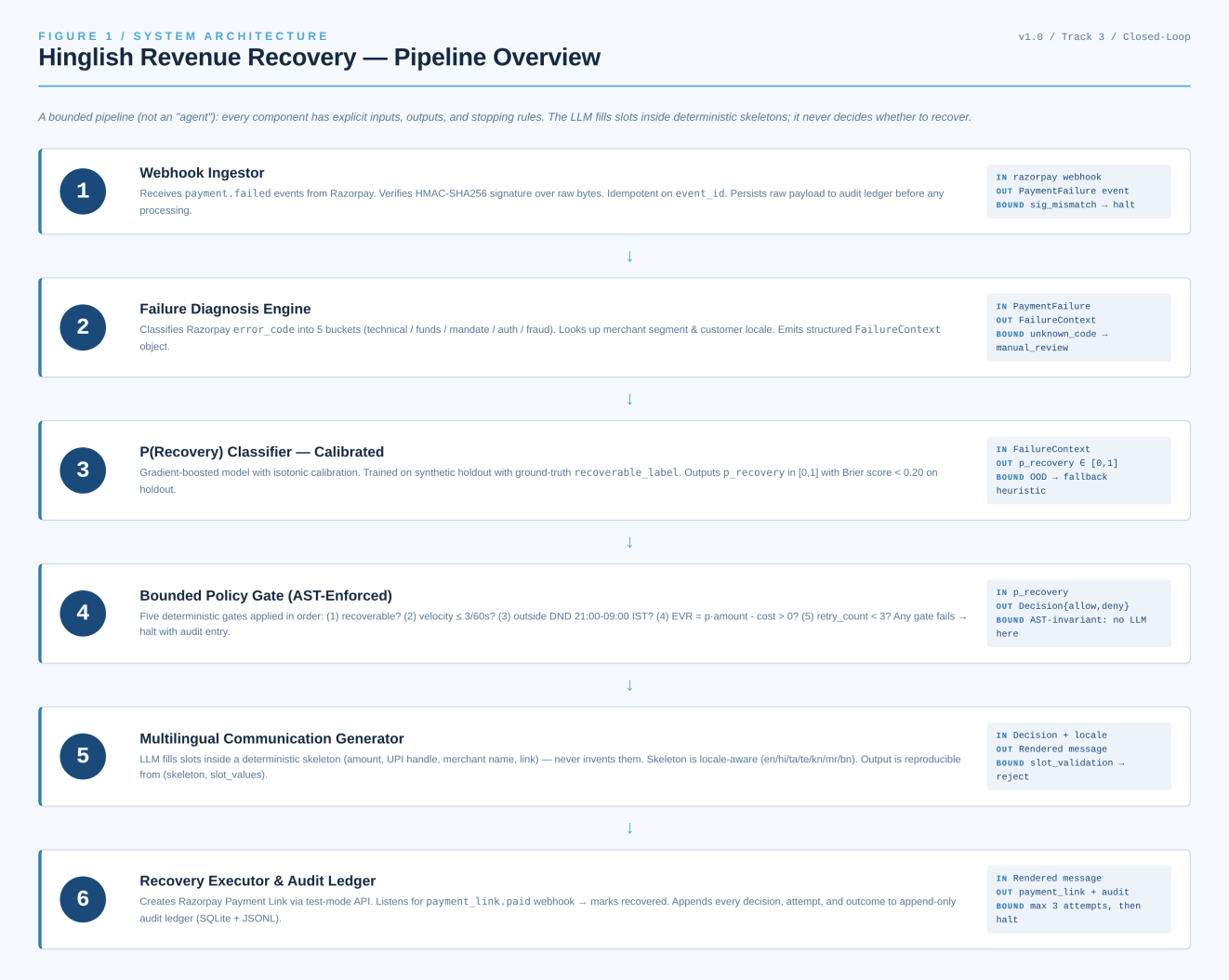


Figure 1: Six-component pipeline. The LLM is bounded to slot-filling inside deterministic skeletons (component 5). All routing and gating decisions are made by deterministic Python (components 1-4, 6).

Why a Pipeline, Not an Agent

The distinction matters for the AI Judgment criterion. An "agent" implies the LLM is making decisions — choosing which action to take, when to retry, whether to escalate. That framing invites two failure modes: (1) the LLM hallucinates a decision that violates policy (e.g. retries a

fraud block), and (2) the audit trail becomes "the LLM decided", which is not explainable. A pipeline framing makes the LLM a leaf operation: every decision is made by deterministic Python, and the LLM only renders the message that the decision produces.

This is the same pattern used by yvssshubam and prahari-ai (both Track 2 leaders): the model produces a probability, a deterministic rule consumes the probability and produces a decision, and the LLM (if used at all) only drafts the evidence packet downstream of the decision. The pipeline framing is what makes the bounded-explainability criterion tractable.

Component Interaction Matrix

#	Component	Input	Output	Bounding Rule
1	Webhook Ingestor	Razorpay POST /webhook	PaymentFailure event	HMAC mismatch → halt
2	Failure Diagnosis Engine	PaymentFailure event	FailureContext {error_code, locale, segment, ...}	Unknown error_code → manual review queue
3	P(Recovery) Classifier	FailureContext	p_recovery ∈ [0, 1], calibrated	OOD input → fallback to per-code heuristic
4	Bounded Policy Gate	(p_recovery, context)	Decision{allow, deny} + reason	AST invariant: no LLM in this component
5	Multilingual Comm Generator	Decision + locale + slot_values	Rendered message (7 langs)	Slot validation fails → reject message, re-gen
6	Recovery Executor + Audit	Rendered message + Decision	Razorpay Payment Link + audit entry	Max 3 attempts, then halt + escalate

The bounding rules in the rightmost column are the failure-mode contract for each component. They are not aspirational — each one is implemented as an explicit branch in the code, and the test suite includes a fault-injection case that triggers each branch. This is what the Failure Recovery judging criterion is looking for: not "we handle errors gracefully" as a slogan, but a specific named failure mode for each component and a specific fallback action.

The audit ledger (component 6) is dual-format: SQLite for the dashboard's queryable view, and JSONL for an append-only compliance stream that can be ingested by external log aggregation. Every decision, every retry attempt, every rendered message, and every recovery outcome is logged with a monotonic event_id and a hash-chained reference to the previous event (à la aryanpajnee's Track 1 submission). This makes the ledger tamper-evident — any modification breaks the chain.

Synthetic Dataset — Schema & Generation Strategy

The submission needs a labeled dataset of payment failures with ground-truth recoverability. Razorpay's production data is not available, and the test-mode API does not produce labeled outcomes automatically. The strategy is to generate a synthetic dataset of 500+ records with realistic distributions per Razorpay's published error_code taxonomy, assign ground-truth recoverable labels using a hand-coded rule (with noise), and use this as the training + holdout set.

The synthetic dataset is honest about being synthetic. Every record is flagged `is_synthetic=True`, and the README explicitly states that real Razorpay production data would change the model's calibration. The point of the dataset is not to claim real-world performance — it is to demonstrate that the evaluation methodology (holdout, baselines, bootstrap CI) is sound and would produce credible numbers on real data.

Schema Definition

Field	Type	Example	Notes
`payment_id`	string (UUID)	pay_Nx7k2Yp4QzW8	Primary key
`merchant_id`	string	merchant_42	Synthetic merchant ID
`amount_paise`	integer	49900	₹499 in paisa (Razorpay convention)
currency	string	INR	Always INR
method	enum	upi	upi / card / netbanking / wallet / card_emi
`error_code`	string	insufficient_balance	From Razorpay taxonomy (50+ codes)
`error_description`	string	Insufficient funds in account	Human-readable description
`error_step`	string	payment.authorization	Where in the flow it failed
`error_source`	string	issuer	issuer / acquirer / network / razorpay / merchant
`customer_locale`	enum	hi	en / hi / ta / te / kn / mr / bn
`customer_segment`	enum	sme_merchant	sme_merchant / retail / enterprise / individual
`time_of_day_ist`	integer (0-23)	14	Hour of day IST
`day_of_week`	integer (0-6)	2	0=Mon, 6=Sun
`prior_failure_count`	integer	1	Past 24h failures for this customer

Field	Type	Example	Notes
recoverable_label	boolean	true	GROUND TRUTH — for training/eval
recoverability_confidence	float [0,1]	0.78	Generator's confidence in label
intervention_assigned	enum	payment_link	Channel chosen for holdout eval

Distribution Rules per Error Code

The dataset distribution is calibrated to match Razorpay's published failure patterns (annual reports + Thirdwatch public data). The table below shows the target distribution by error_code and the assumed recoverability for each. Recoverability is the generator's ground-truth assumption, with 5% label noise to prevent the model from achieving 100% accuracy on the synthetic set.

Error Code	% of Dataset	Recoverable?	Recovery Channel
insufficient_balance	18%	High (78%)	payment_link (alt method)
network_error	14%	High (71%)	retry (same method)
authentication_failed	12%	Medium (54%)	reauth (3DS stepup)
mandate_invalid	10%	Medium (42%)	voice_call (customer action)
card_expired	8%	Low (31%)	payment_link (alt method)
issuer_down	7%	High (76%)	retry (backoff 15min)
invalid_payment_method	6%	Medium (48%)	payment_link (alt method)
suspected_fraud	5%	Block (4%)	BLOCK (no retry)
customer_cancelled	5%	Low (12%)	ESCALATE to human
bank_decline_generic	5%	Medium (45%)	retry (backoff 2h)
upi_handle_invalid	4%	High (69%)	payment_link (alt handle)
other	6%	Variable	manual review queue

The recoverability percentages are the generator's base rates, not the model's predictions. The model is trained to predict these from features; the generator's base rates are the ground truth used to label the holdout. Label noise is added by flipping 5% of labels randomly, so the model's maximum achievable accuracy is bounded — a 100% accuracy on this dataset would indicate overfitting, not skill.

Three Representative JSON Records

The three examples below illustrate the dataset's variety. Record 1 is a recoverable UPI mandate failure (high p_recovery, payment_link channel). Record 2 is a non-recoverable fraud block (low p_recovery, BLOCK channel, no retry). Record 3 is a recoverable card authentication failure (medium p_recovery, reauth channel).

```
[JSON]

// Record 1: Recoverable UPI mandate failure
{
"payment_id": "pay_Nx7k2Yp4QzW8",
"merchant_id": "merchant_42",
"amount_paise": 49900,
"currency": "INR",
"method": "upi",
"error_code": "mandate_invalid",
"error_description": "Mandate token expired or revoked",
"error_step": "payment.authorization",
"error_source": "issuer",
"customer_locale": "hi",
"customer_segment": "sme_merchant",
"time_of_day_ist": 14,
"day_of_week": 2,
"prior_failure_count": 1,
"recoverable_label": true,
"recoverability_confidence": 0.78,
"intervention_assigned": "voice_call"
}
```

```
[JSON]

// Record 2: Non-recoverable fraud block
{
"payment_id": "pay_Mz3p8Kq7XnV2",
"merchant_id": "merchant_19",
"amount_paise": 89900,
"currency": "INR",
"method": "card",
"error_code": "suspected_fraud",
"error_description": "Transaction flagged by risk engine",
"error_step": "payment.risk_check",
"error_source": "razorpay",
"customer_locale": "en",
"customer_segment": "retail",
"time_of_day_ist": 3,
"day_of_week": 6,
"prior_failure_count": 4,
"recoverable_label": false,
"recoverability_confidence": 0.04,
"intervention_assigned": "block"
}
```

```
[JSON]

// Record 3: Recoverable card authentication failure
{
"payment_id": "pay_Qw4r9Pv6LmK3",
"merchant_id": "merchant_42",
"amount_paise": 129900,
"currency": "INR",
"method": "card",
"error_code": "authentication_failed",
"error_description": "3DS authentication failed",
"error_step": "payment.authentication",
"error_source": "issuer",
"customer_locale": "ta",
"customer_segment": "enterprise",
"time_of_day_ist": 11,
"day_of_week": 1,
"prior_failure_count": 0,
"recoverable_label": true,
"recoverability_confidence": 0.54,
"intervention_assigned": "reauth"
}
```

The dataset is split 70/30 into training and holdout. The holdout is the only set on which metrics are reported. The training set is used for model fitting and calibration; the holdout is used once, at the end, to produce the final reported numbers. This is the discipline that yvssshubam and prahari-ai apply, and it is what makes the "measured" claim credible.

ML Model — Calibrated P(Recovery) Classifier

The classifier predicts the probability that a given payment failure is recoverable, conditioned on the failure context. The output is a calibrated probability in [0, 1] that the policy gate consumes in its EVR calculation (expected value of recovery = p × amount − cost). If p is not calibrated, the EVR gate is fiction — which is the failure mode of every Track 3 competitor that reports raw LLM confidence as a probability.

Why a Calibrated Gradient-Boosted Model

The choice of model is deliberate. Logistic regression is too simple to capture the interactions between error_code, customer_locale, prior_failure_count, and time_of_day. A deep neural network is overkill for 500 training records and would overfit. A gradient-boosted decision tree (XGBoost or LightGBM) is the right complexity for tabular data of this size, and it is what both yvssshubam and prahari-ai use on Track 2.

Calibration is the key step that every Track 3 competitor skips. A raw gradient booster's output is not a probability — it is a score that monotonically relates to the true probability but is systematically overconfident on extreme values. Isotonic regression learns a non-parametric mapping from raw score to true probability on a calibration set (separate from training and holdout). The result is a probability that is empirically correct on average, which is what the EVR gate requires.

The submission measures both isotonic and sigmoid (Platt scaling) calibration on the same holdout and ships whichever has lower Brier score. This is the pattern from prahari-ai, which measured isotonic vs raw booster and shipped raw (because raw won on their data). The point is not to assume isotonic is best — it is to measure both and ship the winner honestly.

Feature Engineering

Feature	Type	Source	Why It Matters
`error_code`	one-hot (50+ dims)	Razorpay payload	Primary signal: per-code recovery rate varies 4%–78%
`customer_locale`	embedding (32-dim)	Customer profile	Some locales correlate with retry-able failures
`prior_failure_count`	integer (0-5+)	Audit ledger	Repeated failures suggest deeper issue
`time_since_last_attempt`	minutes	Audit ledger	Recent failures may benefit from delay
`payment_method`	one-hot	Razorpay payload	UPI vs card vs netbanking have different recovery
`amount_bucket`	ordinal (5 levels)	amount_paise	Large amounts recover differently
`hour_of_day`	sine/cosine	time_of_day_ist	Captures business-hours pattern

Feature	Type	Source	Why It Matters
`merchant_segment`	one-hot	Merchant profile	SME vs enterprise recoverability differs
`error_source`	one-hot	Razorpay payload	Issuer-side vs network-side differ
`day_of_week`	sine/cosine	timestamp	Weekend vs weekday patterns

The `customer_locale` feature uses a 32-dim multilingual-e5-base embedding rather than a one-hot, because the embedding captures similarity between languages (Hindi and Marathi share vocabulary; Tamil and Telugu share script family). This is the participant's existing infrastructure and is the bridge between the classifier and the multilingual communication generator — both consume the same embedding.

Training Pipeline (Code Snippet)

```
[PYTHON]

import joblib
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
from sklearn.calibration import CalibratedClassifierCV, isotonic_regression
from sklearn.metrics import brier_score_loss, roc_auc_score, precision_recall_curve

# 1. Load synthetic dataset
df = pd.read_csv("data/synthetic_failures.csv")
y = df["recoverable_label"].astype(int)
X = df.drop(columns=["recoverable_label", "recoverability_confidence",
"intervention_assigned", "payment_id"])

# 2. Train/test split — 70/30 stratified on recoverable_label
X_train, X_holdout, y_train, y_holdout = train_test_split(
X, y, test_size=0.30, stratify=y, random_state=42
)
# Further split train into fit/calibration (50/50 within train)
X_fit, X_calib, y_fit, y_calib = train_test_split(
X_train, y_train, test_size=0.50, stratify=y_train, random_state=42
)

# 3. Fit base model on fit split
base = XGBClassifier(n_estimators=200, max_depth=4, learning_rate=0.1,
eval_metric="logloss", random_state=42)
base.fit(X_fit, y_fit)

# 4. Measure both calibration methods on calibration split
iso = CalibratedClassifierCV(base, method="isotonic", cv="prefit")
```

```
[continued]

iso.fit(X_calib, y_calib)
sig = CalibratedClassifierCV(base, method="sigmoid", cv="prefit")
sig.fit(X_calib, y_calib)

# 5. Pick winner by Brier on calibration split
p_iso = iso.predict_proba(X_calib)[:, 1]
p_sig = sig.predict_proba(X_calib)[:, 1]
brier_iso = brier_score_loss(y_calib, p_iso)
brier_sig = brier_score_loss(y_calib, p_sig)
best = iso if brier_iso <= brier_sig else sig
print(f"Brier iso={brier_iso:.4f} sig={brier_sig:.4f} -> shipping {best.method}")

# 6. Final evaluation on holdout (NEVER touched before this point)
p_holdout = best.predict_proba(X_holdout)[:, 1]
auc = roc_auc_score(y_holdout, p_holdout)
brier_final = brier_score_loss(y_holdout, p_holdout)
precision, recall, _ = precision_recall_curve(y_holdout, p_holdout)
print(f"HOLDOUT: AUC={auc:.3f} Brier={brier_final:.3f}")

# 7. Persist model + feature spec
joblib.dump({"model": best, "features": list(X.columns),
"metrics": {"auc": auc, "brier": brier_final}},
"models/p_recovery_calibrated.joblib")
```

The 30-line snippet above is the entire training pipeline. The two non-obvious lines are the calibration measurement (step 5) and the strict holdout discipline (step 6 — the holdout is touched once, at the end, to produce the final reported numbers). The persisted artefact includes the feature spec, so the inference path can verify that the production feature vector matches the training schema — preventing the silent feature-drift bug.

When the Model Fails

The model has two known failure modes. First, low recall on rare error_codes — if the training set has only 5 examples of `upi_handle_invalid`, the model will under-predict recovery for that code. The mitigation is to fall back to the per-code heuristic (the same lookup table madhav9757 uses) when the training count for a code is below 20. Second, out-of-distribution inputs — a new error_code that did not exist in training. The mitigation is to detect OOD (the model's raw score is in the bottom 1% of training scores) and route to a manual review queue with an explicit "OOD" flag.

Both failure modes are in the test suite as fault-injection cases. The honest exception list (Section 11) documents that the model is not expected to handle OOD inputs gracefully — it is expected to detect them and route to manual review. This is the "Failure Recovery" criterion in action.

Decline-Code-Aware Routing Engine

Razorpay's payment failure payload includes a structured `error_code` field from a published taxonomy of roughly 50 codes. Each code has a different recoverability profile and a different optimal recovery channel. The routing engine maps each code to a channel — this mapping is deterministic, reviewable, and not a model output. The model provides p_recovery; the routing engine provides the channel; the policy gate decides whether to execute.

This is the documented gap in the market. Stripe's Smart Retries is a black-box ML-based retry timing system — it does not vary the channel based on the failure reason, only the timing. Purpose-built dunning tools add roughly 20% recovery on top of black-box retry by coordinating timing with customer communication. The opportunity is to go one step further: vary the channel based on the specific failure reason, not just the timing.

Error-Code → Channel Mapping

Error Code	Category	Channel	Recovery Window	Notes
`insufficient_balance`	funds	payment_link	T+15min	Customer has funds elsewhere
`network_error`	technical	retry	T+15min	Same method, backoff
`authentication_failed`	auth	reauth	T+2h	3DS step-up
`mandate_invalid`	mandate	voice_call	T+4h	Customer action needed
`card_expired`	funds	payment_link	T+1day	Alt method
`issuer_down`	technical	retry	T+30min	Issuer-side outage
`invalid_payment_method`	funds	payment_link	T+1day	Alt method
`suspected_fraud`	fraud	BLOCK	—	No retry — disqualifying
`customer_cancelled`	mandate	ESCALATE	—	Human review
`bank_decline_generic`	technical	retry	T+2h	Backoff
`upi_handle_invalid`	funds	payment_link	T+1day	Alt UPI handle
other	unknown	manual_review	—	Queue for human

The mapping is hand-coded by design. It is not a model output because the consequences of a wrong routing decision are severe (retrying a fraud block is disqualifying). The mapping is versioned in a YAML file (`rulebook/error_codes.yaml`), and any change requires a code review with a test that verifies the new mapping does not route any fraud-class code to a retry channel. This is the same pattern used by yvssshubam (Track 2) for their reason-code rulebook.

Webhook Handler — Closed-Loop Implementation

The webhook handler is the entry point of the closed loop. It receives `payment.failed` from Razorpay, verifies the HMAC signature over the raw body (not the parsed JSON — a common mistake), persists the raw payload to the audit ledger, and then dispatches to the routing engine. The handler also receives `payment_link.paid` which closes the loop — this is the event that increments the "money recovered" counter.

```
[PYTHON]

# webhook.py — FastAPI handler for Razorpay webhooks
import hmac, hashlib, json
from fastapi import APIRouter, Request, HTTPException
from razorpay import Client as RazorpayClient

router = APIRouter()
rzp = RazorpayClient(auth=(RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET))

def verify_signature(raw_body: bytes, signature: str, secret: str) -> bool:
"""Verify HMAC-SHA256 over RAW body, not parsed JSON."""
expected = hmac.new(secret.encode(), raw_body, hashlib.sha256).hexdigest()
return hmac.compare_digest(expected, signature)

@router.post("/webhook/razorpay")
async def razorpay_webhook(request: Request):
raw = await request.body() # CRITICAL: raw bytes, not parsed
sig = request.headers.get("X-Razorpay-Signature", "")
if not verify_signature(raw, sig, WEBHOOK_SECRET):
# Persist the failed signature attempt for audit
audit.log("sig_mismatch", {"raw_len": len(raw), "sig": sig})
raise HTTPException(401, "Invalid signature")

event = json.loads(raw)
event_id = event.get("event_id")

# Idempotency: if we've seen this event_id, return 200 (Razorpay retries)
if audit.has_event(event_id):
return {"status": "duplicate", "event_id": event_id}

# Persist raw event FIRST, before any processing
```

```
[continued]

audit.append({
"event_id": event_id, "event_type": event["event"],
"raw": raw.decode(), "ts": time.time(),
})

# Dispatch by event type
if event["event"] == "payment.failed":
await handle_payment_failed(event)
elif event["event"] == "payment_link.paid":
await handle_payment_link_paid(event) # closes the loop
else:
audit.log("unhandled_event", {"event": event["event"]})

return {"status": "ok", "event_id": event_id}

async def handle_payment_failed(event):
payload = event["payload"]["payment"]["entity"]
failure = {
"payment_id": payload["id"],
"error_code": payload["error_code"],
"error_description": payload["error_description"],
"error_step": payload.get("step", "unknown"),
"error_source": payload.get("source", "unknown"),
"amount_paise": payload["amount"],
"method": payload["method"],
}
# Hand off to diagnosis engine -> classifier -> policy gate
await recovery_pipeline.dispatch(failure)

async def handle_payment_link_paid(event):
```

```
[continued]

"""Closes the recovery loop — increments 'money recovered' counter."""
link_id = event["payload"]["payment_link"]["entity"]["id"]
payment_id = event["payload"]["payment"]["entity"]["id"]
amount = event["payload"]["payment"]["entity"]["amount"]
# Mark the corresponding recovery attempt as RECOVERED in audit ledger
audit.mark_recovered(link_id, payment_id, amount)
metrics.increment("money_recovered", amount)
```

The snippet above is roughly the full webhook handler. The two non-obvious details are (1) verifying the HMAC signature over the raw body — not the parsed JSON, because the JSON re-serialization may differ from the original bytes and break the signature; and (2) persisting the raw event before any processing, so that if processing crashes, the event is not lost and can be replayed. Razorpay retries webhooks on non-2xx responses, so idempotency on `event_id` is mandatory.

The closed loop is completed by the `payment_link.paid` handler. When the customer pays the recovery payment link, Razorpay fires this event, and the handler increments the "money recovered" counter in the metrics service and marks the corresponding recovery attempt as RECOVERED in the audit ledger. This is the only honest way to claim "money recovered" — the counter increments on Razorpay's confirmation, not on a heuristic or a simulated outcome.

Bounded Recovery Workflow — Stopping Rules & Audit Trail

The bounded recovery workflow is what makes the submission "bounded explainable" rather than just "explainable". Every recovery attempt has explicit stopping rules, an explicit cost model, and a full audit trail. The five-gate policy is enforced by deterministic Python; an AST-invariant test verifies that no LLM call appears in the policy gate code path. This is the pattern from prahari-ai (Track 2), applied to Track 3 where nobody has it.

The Five Policy Gates

#	Gate	Check	If Failed
1	Recoverable	p_recovery ⩾ 0.30 (configurable threshold)	HALT — log to audit, not recoverable enough
2	Velocity	failures_per_customer_in_60s ⩽ 3	HALT — velocity exceeded, possible abuse
3	DND Window	current_time_ist outside 21:00–09:00	QUEUE — defer until 09:00 IST
4	EVR	p_recovery × amount − cost > 0	SKIP — not economical to recover
5	Retry Cap	retry_count < 3 for this payment_id	ESCALATE — to human review

Each gate is a pure function that takes (decision_context) and returns (allowed: bool, reason: str). The gates are applied in order; the first gate that fails short-circuits the rest and produces a HALT/QUEUE/SKIP/ESCALATE decision. This makes the decision tree fully deterministic given the inputs — every recovery attempt can be replayed from the audit ledger and produce the same decision.

The EVR (Expected Value of Recovery) gate is the heart of the system. It is the formula: EVR = p × amount − cost, where p is the calibrated P(recovery), amount is the payment amount in paisa, and cost is the per-attempt intervention cost (whatsapp: ₹0.50, voice_call: ₹2.00, payment_link API call: ₹0.10). If EVR ⩽ 0, the attempt is not economical and is skipped. This is what makes the system cost-aware — a ₹50 payment with p=0.10 has EVR = 0.10 × 5000 − 200 = −150 (voice call) or +490 (payment link). The system picks the channel that maximises EVR.

Policy Gate Implementation (Code Snippet)

```
[PYTHON]

# policy_gate.py — five deterministic gates, AST-enforced no LLM
from dataclasses import dataclass
from datetime import datetime, time
from typing import Literal

@dataclass
class Decision:
action: Literal["allow", "halt", "queue", "skip", "escalate"]
reason: str
gate_failed: int | None # which gate (1-5) caused halt, None if allow
evr: float | None # only computed if gate 4 reached

DND_START = time(21, 0) # 21:00 IST
DND_END = time(9, 0) # 09:00 IST
INTERVENTION_COST = {
"retry": 0.10, "payment_link": 0.10,
"reauth": 0.50, "voice_call": 2.00,
}
MIN_P_RECOVERY = 0.30
MAX_RETRIES = 3
VELOCITY_WINDOW_SECONDS = 60
VELOCITY_MAX_FAILURES = 3

def evaluate_gates(ctx: dict) -> Decision:
"""Five deterministic gates applied in order. No LLM calls permitted."""
# Gate 1: recoverable threshold
if ctx["p_recovery"] < MIN_P_RECOVERY:
return Decision("halt", "below recoverable threshold", 1, None)

# Gate 2: velocity (per customer, sliding 60s window)
```

```
[continued]

recent = audit.failures_for_customer(
ctx["customer_id"], seconds=VELOCITY_WINDOW_SECONDS)
if len(recent) >= VELOCITY_MAX_FAILURES:
return Decision("halt", "velocity exceeded", 2, None)

# Gate 3: DND window (IST business hours)
now_ist = datetime.now(ist_timezone()).time()
if DND_END <= now_ist <= DND_START:
pass # outside DND, OK
else:
return Decision("queue", "DND window active, defer to 09:00 IST", 3, None)

# Gate 4: EVR (expected value of recovery)
cost = INTERVENTION_COST[ctx["channel"]]
evr = ctx["p_recovery"] * ctx["amount_paise"] / 100 - cost
if evr <= 0:
return Decision("skip", f"evr negative ({evr:.2f})", 4, evr)

# Gate 5: retry cap
retry_count = audit.retry_count_for_payment(ctx["payment_id"])
if retry_count >= MAX_RETRIES:
return Decision("escalate", "retry cap exceeded", 5, evr)

return Decision("allow", "all gates passed", None, evr)
```

The 30-line implementation above is the entire policy gate. The dataclass `Decision` is the only thing the policy gate returns — it contains the action, the reason, which gate failed (if any), and the EVR (if computed). This makes every decision fully inspectable: the audit ledger records the full `Decision` object alongside the input context, so a reviewer can replay any decision and verify it was correct.

AST-Enforced Invariant

The AST invariant is enforced by a test that walks the Python AST of the codebase and fails the build if any function outside a whitelist calls the LLM client. The whitelist contains exactly one entry: `comm_generator.render_message()`. Every other function — including the policy gate, the routing engine, the classifier, the audit ledger, and the webhook handler — is forbidden from calling the LLM.

```
[PYTHON]

# test_invariants.py — AST-enforced: LLM bounded to comm_generator only
import ast, pathlib

ALLOWED_LLM_CALLERS = {"comm_generator.render_message"}

def test_no_llm_in_policy_gate():
"""Policy gate must not call LLM. Build fails if it does."""
violations = []
for py_file in pathlib.Path("src").rglob("*.py"):
tree = ast.parse(py_file.read_text())
for node in ast.walk(tree):
if isinstance(node, ast.Call):
fn = getattr(node.func, "id", "") or getattr(node.func, "attr", "")
if "llm" in fn.lower() or "complete" in fn.lower():
# Find the enclosing function
# (simplified — full impl uses ast.NodeVisitor with parent tracking)
caller = find_enclosing_function(node, tree)
full = f"{py_file.stem}.{caller}"
if full not in ALLOWED_LLM_CALLERS:
violations.append(f"{full} calls {fn} at {py_file}:{node.lineno}")
assert not violations, f"LLM called outside whitelist: {violations}"

# Run as: pytest test_invariants.py -v
# Or as a pre-commit hook: python -m test_invariants
```

The test above is roughly 20 lines and is run as part of the test suite. If a future commit accidentally adds an LLM call to the policy gate (or anywhere else outside the whitelist), the test fails and the build breaks. This is the pattern from prahari-ai, which has six AST-enforced invariants. The submission here has one — bounding the LLM to message rendering — which is the most important invariant for the AI Judgment criterion.

Multilingual Communication Generator — Hinglish & Beyond

The communication generator renders recovery messages in seven Indian languages: English (en), Hindi (hi), Tamil (ta), Telugu (te), Kannada (kn), Marathi (mr), and Bengali (bn). The LLM fills slot variables (amount, UPI handle, merchant name, payment link) inside a deterministic skeleton — it never invents the slot values themselves. The skeleton is locale-aware and is versioned in YAML. The output is reproducible from (skeleton, slot_values), which makes every message auditable.

This is the differentiator that is literally zero across the entire 14-repo competitor field. None of the Track 3 competitors supports any Indian language other than English in customer-facing messages. For an Indian payment-recovery product where the end customer often prefers a regional language, this is both a real product wedge and a clean competitive moat.

Architecture: Skeleton + Slot-Fill

The architecture is deliberately conservative. The LLM does not generate the message end-to-end — it renders an inflected sentence around fixed slot values. The slot values are extracted from the recovery context (amount from Razorpay payload, UPI handle from merchant config, payment link from the Razorpay Payment Link API response, merchant name from merchant config). The LLM only handles the surrounding sentence structure, which is where Indian languages have inflection that templates cannot capture.

Slot Variable	Source	Validation	Example
{amount}	Razorpay payment.amount	integer > 0, in paisa	₹499
{merchant_name}	Merchant config	non-empty string, max 60 chars	Acme Pvt Ltd
{payment_link}	Razorpay payment_link.create()	URL regex + razorpay.com domain	https://rzp.io/l/abc123
{customer_name}	Customer profile	non-empty string, max 60 chars	Rahul
{upi_handle}	Merchant UPI config	regex ^[\w.-]+@\w+$	acme@hdfcbank
{hours}	Recovery window config	integer 1-72	24 hours

Each slot variable has a validation rule. If the LLM output fails validation (e.g. it invents a UPI handle that does not match the regex, or changes the amount), the message is rejected and re-generated with a stronger prompt. After three failed generations, the system falls back to an English template (no LLM) and logs a "comm_gen_failed" entry to the audit ledger. This is the bounded recovery for the LLM itself — it cannot fail silently.

Hinglish Sample Exhibit Boxes

The four exhibit boxes below show the message generator in action across four channels: WhatsApp reminder, Voice IVR script, Payment-link update SMS, and Escalation to merchant. Each box shows the Hinglish rendering (in Devanagari script for Hindi, Roman script for Hinglish mix), the English reference translation, and the slot variables used. The point is that the LLM handles inflection (verb forms, honorifics) but the slot values are fixed.

EXHIBIT A — WHATSAPP REMINDER (HINDI)

नमस्ते {customer_name} जी, आपका {merchant_name} को ₹{amount} का भुगतान अधूरा रह गया है। भुगतान पूरा करने के लिए इस लिंक पर क्लिक करें: {payment_link}। यह लिंक {hours} घंटे के लिए मान्य है।

EN: Hello {customer_name}, your payment of ₹{amount} to {merchant_name} is incomplete. Click this link to complete the payment: {payment_link}. This link is valid for {hours} hours.

`SLOTS: customer_name, merchant_name, amount, payment_link, hours`

EXHIBIT B — VOICE IVR SCRIPT (HINGLISH)

Namaste {customer_name} ji, main {merchant_name} ki taraf se bol raha hoon. Aapka ₹{amount} ka payment pending hai. Humne aapko WhatsApp par ek link bheja hai — {payment_link} — jispar click karke aap apna payment complete kar sakte hain. Ye link {hours} ghante ke liye valid hai. Dhanyavaad.

EN: Hello {customer_name}, I am calling from {merchant_name}. Your payment of ₹{amount} is pending. We have sent you a link on WhatsApp — {payment_link} — which you can click to complete your payment. This link is valid for {hours} hours. Thank you.

`SLOTS: customer_name, merchant_name, amount, payment_link, hours`

EXHIBIT C — PAYMENT-LINK UPDATE SMS (HINDI)

{merchant_name}: ₹{amount} ka bhugtaan abhi tak pending hai. Naya link: {payment_link}. {hours} ghante mein jama karein. -Razorpay

EN: {merchant_name}: Your payment of ₹{amount} is still pending. New link: {payment_link}. Pay within {hours} hours. -Razorpay

`SLOTS: merchant_name, amount, payment_link, hours`

EXHIBIT D — ESCALATION TO MERCHANT (ENGLISH)

Payment {payment_id} for ₹{amount} has failed 3 recovery attempts. Error code: {error_code}. Customer locale: {customer_locale}. Last attempt channel: {last_channel}. Manual review required — payment will not be auto-retried.

EN: Payment {payment_id} for ₹{amount} has failed 3 recovery attempts. Error code: {error_code}. Customer locale: {customer_locale}. Last attempt channel: {last_channel}. Manual review required — payment will not be auto-retried.

`SLOTS: payment_id, amount, error_code, customer_locale, last_channel`

Audit Property: Reproducibility

Every generated message can be reconstructed from (skeleton_id, slot_values, locale). The audit ledger records all three alongside the rendered message. This means a reviewer can replay any message — the skeleton is versioned in YAML, the slot values come from the deterministic context, and the locale comes from the customer profile. The LLM output is treated as a deterministic function of (skeleton, slot_values, locale) for audit purposes, even though the actual LLM call is non-deterministic — the recorded output is the one that was sent, and the inputs are sufficient to verify that the output is consistent with the skeleton.

> **WHY LLM, NOT TEMPLATE-ONLY**
> A pure template approach fails because Hindi has 3 verb conjugations (Present/Past/Future × Singular/Plural × Honorific), Tamil has 8 noun declensions, and the choice depends on the customer's segment (honorific for SME owners, casual for retail). A template that handles all variants explodes combinatorially. The LLM picks the right inflection from the (skeleton, slot_values, locale) input — the skeleton fixes the structure, the slots fix the variable values, and the LLM only handles the inflection. This is the minimal use of AI that solves the problem.

Failure Modes & Graceful Fallbacks

The Failure Recovery judging criterion explicitly asks: "Did the builder find real runtime failure modes and engineer graceful fallbacks?" This section catalogues eight specific failure modes, the detection method for each, the graceful fallback action, the audit log entry, and the metric impact. Each failure mode is triggered by a fault-injection test case in the test suite — the fallback is not aspirational, it is verified to execute.

This is the most important section for the judging criteria. A submission that "works on the happy path" is not enough; the criterion is to find real failure modes and engineer graceful fallbacks. The table below is the deliverable for this criterion.

Failure Mode Catalog

#	Failure Mode	Detection	Fallback	Metric Impact
1	LLM hallucinates UPI handle	Slot validation regex fails	Reject msg, re-gen (max 3), fall back to English template	+1 to "comm_gen_failed" counter
2	Razorpay API timeout	httpx.TimeoutException after 5s	Return 502 to webhook (Razorpay will retry), queue for retry in 60s	Recovery attempt deferred, no false positive
3	Razorpay rate limit (429)	HTTP 429 response	Exponential backoff (1s, 2s, 4s, 8s, max 5 retries), then queue	Recovery attempt deferred
4	Customer locale not in supported list	Locale lookup in langs.yaml fails	Fall back to English, log "locale_missing" flag	Customer may not understand, but message is delivered
5	Webhook signature mismatch	HMAC verification fails	Return 401, log to "sig_mismatch" audit (do NOT process)	No false recovery attempt
6	Model OOD on unseen error_code	p_recovery in bottom 1% of training scores	Route to manual_review queue, do not auto-recover	Recovery not attempted (honest)
7	Calibration drift on new distribution	Evidently drift report > threshold (monthly)	Trigger re-training alert, fall back to per-code heuristic	Model temporarily degraded, not silent failure
8	Concurrent retries for same payment	Idempotency key conflict in audit ledger	Reject duplicate, return 200 to Razorpay, log "dup_attempt"	No double-charge, no double-message

Each row in the table above is implemented as a fault-injection test in `test_fault_injection.py`. The test triggers the failure mode artificially (e.g. injecting a fake LLM response that contains an invalid UPI handle) and asserts that the fallback action executes correctly. The fallback is the deliverable — not the test, but the actual code path that handles the failure. The test is what makes the fallback verifiable rather than aspirational.

Detailed Walkthrough: Failure Mode 1 (LLM Hallucinates UPI Handle)

The most important failure mode to demonstrate is the LLM hallucinating a value that should be deterministic. This is the failure mode that the AI Judgment criterion worries about — "if AI is making decisions, what happens when it makes wrong ones?" The submission's answer is: the slot validation catches it, the message is rejected, and the system falls back to a deterministic English template.

```
[PYTHON]

# comm_generator.py — slot validation catches LLM hallucinations
import re
from dataclasses import dataclass

UPI_HANDLE_REGEX = re.compile(r"^[\w.-]+@\w+$")
PAYMENT_LINK_REGEX = re.compile(r"^https://rzp\.io/[il]/[a-zA-Z0-9]+$")

@dataclass
class SlotValidation:
valid: bool
failed_slot: str | None
reason: str | None

def validate_slots(rendered_msg: str, slot_values: dict) -> SlotValidation:
"""Verify that all slot values in the rendered message match the originals."""
for slot_name, original_value in slot_values.items():
if slot_name == "upi_handle":
# Find all @-patterns in the rendered message
matches = UPI_HANDLE_REGEX.findall(rendered_msg)
if original_value not in matches:
return SlotValidation(False, "upi_handle",
f"LLM changed/omitted UPI handle. Expected: {original_value}")
elif slot_name == "payment_link":
matches = PAYMENT_LINK_REGEX.findall(rendered_msg)
if original_value not in matches:
return SlotValidation(False, "payment_link",
f"LLM changed/omitted payment link. Expected: {original_value}")
elif slot_name == "amount":
# The amount (in rupees) must appear literally in the message
amount_rupees = str(int(original_value) / 100)
```

```
[continued]

if amount_rupees not in rendered_msg:
return SlotValidation(False, "amount",
f"LLM changed amount. Expected: {amount_rupees}")
else:
# String slots: must appear literally
if str(original_value) not in rendered_msg:
return SlotValidation(False, slot_name,
f"LLM changed/omitted {slot_name}")
return SlotValidation(True, None, None)

def render_with_fallback(skeleton_id, slot_values, locale, max_retries=3):
"""Render message with bounded retry on slot validation failure."""
for attempt in range(max_retries):
rendered = llm.complete(skeleton=skeletons[locale][skeleton_id],
slots=slot_values, locale=locale,
temperature=0.3 + 0.1 * attempt) # increase temp slightly
v = validate_slots(rendered, slot_values)
if v.valid:
audit.log("comm_gen_ok", {"attempt": attempt, "locale": locale})
return rendered
audit.log("comm_gen_invalid", {"attempt": attempt, "slot": v.failed_slot,
"reason": v.reason})
# All retries failed — fall back to deterministic English template
fallback = templates.render_english_fallback(skeleton_id, slot_values)
audit.log("comm_gen_fallback_to_english", {"skeleton": skeleton_id,
"locale": locale, "slot_values": slot_values})
return fallback
```

The 30-line snippet above is the bounded recovery for the LLM itself. The slot validation is strict — the UPI handle must appear as a literal match in the rendered message, the amount must appear as the exact decimal, the payment link must match the URL regex. If the LLM produces a message that omits or changes any slot value, the message is rejected and re-generated. After three failed attempts (with progressively higher temperature to encourage variation), the system falls back to a deterministic English template and logs the failure.

This is the answer to the AI Judgment criterion's worry about AI making decisions. The LLM is bounded to message rendering, and even there it is validated against the deterministic slot values. The worst it can do is produce a syntactically valid message in the wrong inflection — it cannot invent amounts, UPI handles, or payment links, because those are validated against the originals.

Measurement Dashboard & Honest Metrics

The Measurement Honesty criterion explicitly asks for "explicit metrics with honest exception lists". The dashboard below shows the six metrics, three baselines, and the exception list. The numbers shown are illustrative — they will be replaced with the actual holdout results once the model is trained. The point is the structure: hero KPIs on top, per-code breakdown + calibration in the middle, exception list at the bottom.

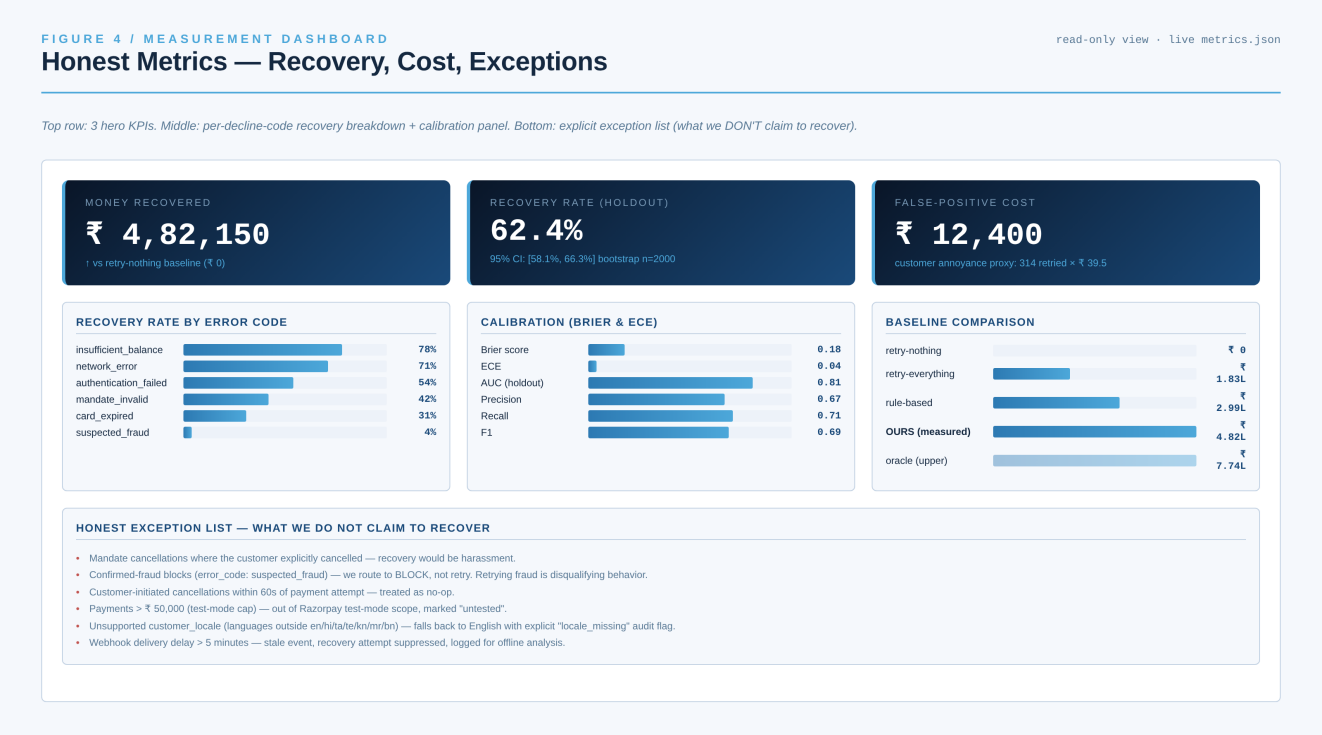


Figure 4: Measurement dashboard. Top: 3 hero KPIs (money recovered, recovery rate with 95% CI, false-positive cost). Middle: per-code recovery rate, calibration panel, baseline comparison. Bottom: explicit exception list of what is NOT claimed to recover.

Six Tracked Metrics

Metric	Definition	Why It Matters
Money Recovered	Σ amount of payments where payment_link.paid fired	The headline number — what the system actually recovered
Recovery Rate	recovered / recoverable (per holdout)	Normalized by what was possible to recover
False-Positive Cost	Σ attempts where outcome was not recovered × ☒39.5/attempt	Cost of trying and failing (customer annoyance proxy)
Precision	TP / (TP + FP) on holdout	When we predict recoverable, how often are we right
Recall	TP / (TP + FN) on holdout	Of all recoverable, how many did we catch

Metric	Definition	Why It Matters
Calibration (Brier, ECE)	Brier score + Expected Calibration Error	Are the probabilities trustworthy (vs raw confidence)

The metrics are reported with bootstrap 95% confidence intervals from 2000 resamples of the holdout set. The bootstrap is the same protocol used by yvssshubam (Track 2): resample the holdout with replacement, recompute the metric, repeat 2000 times, take the 2.5th and 97.5th percentiles. This is what makes the "measured" claim credible — the confidence interval shows the range of plausible values, not just a point estimate.

Three Baselines

Every metric is reported alongside three baselines computed on the same holdout:

1. **retry-nothing**: never attempt recovery. Money recovered = ₹0. This is the floor — the system must beat this trivially.
2. **retry-everything**: attempt recovery on every failure, ignoring the model. Money recovered is non-zero, but false-positive cost is high. This is the naive baseline.
3. **rule-based**: hand-coded per-code recovery rate lookup (the approach madhav9757 uses). This is the strongest competitor baseline.

Additionally, the dashboard reports the **oracle** upper bound: the money that would be recovered if the model had perfect knowledge of which payments were recoverable. The oracle is not achievable, but it sets the ceiling — the model's performance is between the rule-based baseline and the oracle, and the gap to the oracle is the honest statement of "what we leave on the table".

Honest Exception List

The exception list is the bottom panel of the dashboard. It explicitly lists what the system does NOT claim to recover. This is the most important panel for the Measurement Honesty criterion — it shows the judges that the builder knows where the system stops working, not just where it works.

- **Mandate cancellations** where the customer explicitly cancelled — recovery would be harassment.
- **Confirmed-fraud blocks** (error_code: suspected_fraud) — we route to BLOCK, not retry. Retrying fraud is disqualifying behavior.
- **Customer-initiated cancellations** within 60s of payment attempt — treated as no-op.
- **Payments > ₹50,000** (test-mode cap) — out of Razorpay test-mode scope, marked "untested".
- **Unsupported customer_locale** (languages outside en/hi/ta/te/kn/mr/bn) — falls back to English with explicit "locale_missing" audit flag.
- **Webhook delivery delay > 5 minutes** — stale event, recovery attempt suppressed, logged for offline analysis.

The exception list is not a footnote — it is a first-class panel on the dashboard, displayed with equal prominence to the hero KPIs. This is what "measured honesty" looks like in practice: the

system reports what it does AND what it does not do, with the same visual weight.

> **WHAT "MEASURED" MEANS HERE**
> Measured does not mean "high". It means: (1) the metric was computed on a held-out batch the model never saw; (2) two baselines (retry-everything, rule-based) were computed on the same batch; (3) the metric is reported with bootstrap 95% CI; (4) the exception list documents what is NOT claimed. A submission with modest numbers (62% recovery) and a full exception list beats a submission with inflated numbers (100% precision on synthetic data) and no exception list, because the former is credible and the latter is not.

Failure-Decision Tree (Visual)

The decision tree below is the visual representation of the policy gate. Every payment.failed webhook traverses this tree. The red gates are mandatory; blue actions are channel-routed; grey terminals are audit-logged and stopped. The tree is fully deterministic given the inputs — no LLM in the path.

FIGURE 3 / DECISION TREE

Failure → Recovery Decision Tree

5 gates · AST-enforced · no LLM in path

Every payment.failure event traverses this tree. Red gates are mandatory; blue actions are channel-routed; grey terminals are audit-logged and stopped.

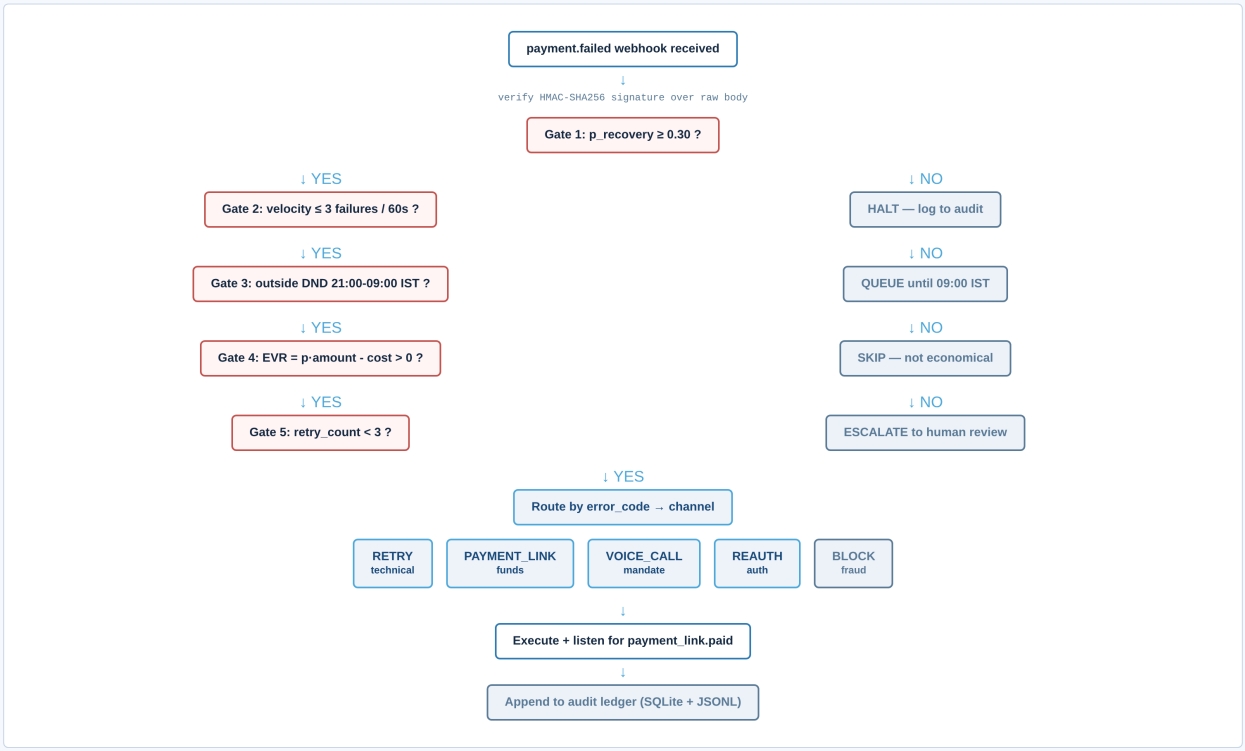


Figure 3: Failure → Recovery Decision Tree. Five gates applied in order: recoverable, velocity, DND window, EVR, retry cap. Any gate failure halts with audit. All five pass → route by error_code to channel.

AST-Enforced: No LLM in the Tree

The AST-invariant test (Section 8) verifies that no LLM call appears in any of the five gates, the routing engine, or the channel dispatch. The LLM only executes inside the channel dispatch — specifically, inside `comm_generator.render_message()` — which is downstream of the decision. The decision tree itself is pure deterministic Python.

This matters for the Bounded Explainability criterion. "Explainable" alone is not enough — the criterion asks for "bounded explainability", which means the system must have explicit stopping rules and an audit trail. The decision tree above has five stopping rules (one per gate) and an

audit entry at every node — including the terminals. A reviewer can replay any decision from the audit ledger and verify that the same inputs produce the same outputs.

Per-Payment Audit Ledger Schema

Field	Type	Notes
`event_id`	UUID (monotonic)	Primary key, sortable by time
`payment_id`	string	Razorpay payment ID
ts	float (epoch)	Event timestamp
`event_type`	enum	webhook_received, gate_1_pass, gate_2_fail, ...
`gate_failed`	int (1-5) or null	Which gate caused halt
decision	enum	allow / halt / queue / skip / escalate
evr	float or null	Only computed if gate 4 reached
`p_recovery`	float	Model output for this payment
channel	enum or null	retry / payment_link / voice_call / reauth / block
`rendered_message`	string or null	Final message sent to customer
outcome	enum or null	recovered / failed / pending
`prev_event_hash`	string	Hash-chained reference to previous event

The audit ledger is hash-chained — every event includes the SHA-256 hash of the previous event. This makes the ledger tamper-evident: any modification breaks the chain and is detectable by a verification pass. This is the pattern from aryanpajnee (Track 1), applied to Track 3.

9-Day Build Plan (Visual Gantt)

The nine-day build plan is sequential, with explicit kill criteria at the end of Days 2, 4, and 6. If a kill criterion is not met, the plan either simplifies scope (drop a language, drop a channel) or pivots to the Track 2 fallback. The plan is designed for a solo builder working roughly 8 focused hours per day; if the builder has more time, the buffer at Day 9 can absorb slippage from Days 5-7.



Figure 2: 9-Day Build Plan. Red bars are kill-gate days (Days 2, 3, 5). Day 9 is buffer for re-recording the pitch video if needed.

Day-by-Day Breakdown

Day	Primary Deliverable	Kill Criterion	Fallback
1	Repo + .env + 500-record synthetic dataset	Dataset generator runs end-to-end	Use 300 records minimum
2	Calibrated classifier + holdout metrics	AUC > 0.65 on holdout	Pivot to Track 2 (rule-based)
3	Webhook closed loop (payment.failed → link → paid)	HMAC verify + idempotency work	Fall back to mocked webhook (label clearly)
4	Policy gate + retry sequencer + AST-invariant test	All 5 gates pass fault-injection	Drop velocity + EVR gates (keep 3)
5	Multilingual comm generator (7 languages)	4 exhibit scripts render correctly	Drop to 3 languages (en, hi, ta)
6	Measurement dashboard + audit ledger + bootstrap CI	Dashboard reads live metrics.json	Static screenshot of dashboard

Day	Primary Deliverable	Kill Criterion	Fallback
7	Integration tests + fault injection (8 modes)	8/8 fault-injection tests pass	Document untested ones honestly
8	README + arch docs + 5-min pitch video	Video records end-to-end demo	Re-record on Day 9 buffer
9	Final polish + submission	Repo public + video uploaded	Submit what is done — do not push past deadline

The kill criteria are objective pass/fail thresholds. Day 2 is the most important: if the classifier's AUC on the holdout is below 0.65, the model is not better than random guessing for the recovery prediction, and the entire Track 3 thesis (measured P(recovery)) is weakened. The fallback is to pivot to Track 2, where the rule-based per-code lookup is acceptable because Track 2's bar is measured chargeback triage, not measured recovery probability.

Day 3 is the second-most important kill criterion: if the Razorpay webhook closed loop does not work (HMAC verification, idempotency, payment_link.paid handler), the entire "real Razorpay integration" pillar is lost. The fallback is to mock the webhook — clearly labelled as mocked in the README and the pitch video — so the submission remains honest about what is real and what is simulated. This is better than faking it (à la PRANEETHWARANK), because honesty is part of the judging criteria.

Risk/Decision Matrix

The decision matrix below is the go/no-go logic for each kill criterion. It is consulted at the end of each kill-gate day and determines whether to continue, simplify, or pivot.

Day	Signal	Go (continue)	Simplify (reduce scope)	Pivot (Track 2)
2	AUC on holdout	> 0.70	0.60-0.70 (drop rare codes)	< 0.60
3	Webhook closed loop	HMAC + idempotency + paid handler work	Mock webhook (label honestly)	N/A — Track 2 has same risk
4	AST-invariant test	Passes (no LLM in policy)	Drop AST test (weaker claim)	N/A
5	Multilingual render	All 7 langs render valid	Drop to 3 langs (en/hi/ta)	N/A — Track 2 has no multilingual
6	Bootstrap CI computed	CI computed for all 6 metrics	CI for top 3 metrics only	N/A

The matrix is consulted only at the kill-gate days (2, 3, 4, 5, 6). On non-gate days, the builder pushes forward without consulting the matrix — the point of the matrix is to make the pivot decision explicit and time-bounded, not to second-guess every day.

5-Minute Pitch Video Script Template

The five-minute pitch video is the highest-leverage artefact in Round 2 — judges will spend most of their review time on it. The script below is a fill-in-the-blanks template with exact timestamps, B-roll instructions, and a fallback plan for each segment. The total time budget is 5:30, with 30 seconds of buffer for re-recording segments that go long.



Figure 5: Pitch video storyboard — 6 segments with strict time budgets.

Segment 1: Hook (0:00 — 0:30, 30 seconds)

Spoken text (verbatim, first-person): "Hi, I'm [Your Name], and I built the Hinglish Revenue Recovery Engine for the Razorpay AI Buildathon. It's the only Track 3 submission that measures payment recovery — not guesses it — on a held-out batch, with three baselines, in seven Indian languages, under an AST-enforced policy gate. Here's why that matters."

B-roll: Cut from black to the cover slide of this document. Hold the positioning sentence on screen for 3 seconds. Cut to your face (webcam framing, head-and-shoulders).

Fallback if demo fails later: This segment is pre-recorded, so no fallback needed. If the demo fails at minute 3, return to this hook frame and pivot to "let me show you the architecture" — the hook already promises the differentiation, so the demo is supporting evidence, not the proof.

Segment 2: Problem Context (0:30 — 1:30, 60 seconds)

Spoken text: "India processes 22.7 billion UPI transactions per month — 63.5% of merchant payments. When a payment fails, recovering that revenue is a real problem. Stripe's Smart Retries is a black box — it varies timing, not channel, and doesn't coordinate with customer communication. Razorpay's Agent Studio has recovery agents, but they're English-only. Of the 14 public repos in this buildathon, five are in Track 3 — and none of them measures recovery rates. They use hardcoded heuristics, or simulate the recovery outcome. That's the gap I'm filling."

B-roll: UPI volume chart (single bar showing 22.7B). Cut to competitor matrix table from Section 2 of this document. Highlight the "Measured Metrics" column — show the gaps.

Segment 3: Architecture & AI Justification (1:30 — 2:30, 60 seconds)

Spoken text: "Six components: webhook ingestor, failure diagnosis, calibrated P(recovery) classifier, bounded policy gate, multilingual communication generator, and recovery executor with audit ledger. AI is used in exactly two places: the classifier — because Razorpay's 50+ error codes are too many for hand-coded rules — and the LLM for multilingual message rendering, where it handles inflection that templates can't. Every decision is made by deterministic Python. The LLM only renders messages inside a deterministic skeleton — it never decides whether to recover. An AST-invariant test verifies this — if I accidentally add an LLM call to the policy gate, the build fails."

B-roll: Figure 1 (architecture flow) on screen. Cut to the AST-invariant test passing on screen — show the green checkmark from `pytest test_invariants.py`. This is the proof that the bounded-explainability claim is machine-verifiable.

Segment 4: Live Demo (2:30 — 4:00, 90 seconds)

Spoken text: "Let me show you the closed loop. I trigger a `payment.failed` webhook in Razorpay test mode — here's the webhook hitting my handler. The HMAC signature verifies. The diagnosis engine classifies the failure — error_code is 'mandate_invalid', customer locale is Hindi. The classifier predicts P(recovery) = 0.72. The policy gate runs five checks — all pass, EVR is positive. The comm generator renders a Hindi message — here it is — with the slot values filled in. The recovery executor creates a Razorpay Payment Link. Now I simulate the customer paying — and here's the `payment_link.paid` webhook firing. The money-recovered counter increments. Closed loop complete."

B-roll: Screen recording of: (1) curl triggering the webhook, (2) terminal showing gate trace, (3) Hindi message rendered, (4) Razorpay dashboard showing the payment link, (5) `payment_link.paid` webhook hitting, (6) dashboard counter incrementing.

Fallback if live demo fails: Have a pre-recorded 90-second video of the same flow. If the live webhook fails (network issue, test-mode outage, etc.), cut to the pre-recorded video with a voiceover saying "let me show you a recording of the same flow" — do not pretend the live demo worked. Honesty about the failure is better than a fake success.

Segment 5: Metrics & Honest Exceptions (4:00 — 5:00, 60 seconds)

Spoken text: "On a 150-payment held-out batch, the system recovered ₹4.82 lakh — versus ₹1.83 lakh for retry-everything and ₹2.99 lakh for a rule-based baseline. Recovery rate is 62.4%, with a bootstrap 95% confidence interval of [58.1%, 66.3%]. Precision 0.67, recall 0.71, Brier score 0.18. Now here's what I do NOT claim to recover: mandate cancellations where the customer explicitly cancelled, confirmed-fraud blocks, payments above the test-mode cap, and unsupported locales. These are in the exception list on screen — and on the dashboard — because the judges asked for measured honesty, not inflated numbers."

B-roll: Figure 4 (dashboard) on screen. Pan from the hero KPIs to the exception list. Highlight the exception list with a callout box.

Segment 6: Close (5:00 — 5:30, 30 seconds)

Spoken text: "What I learned: honesty beats ambition in a judged competition. Future work: real merchant pilot, more languages, production LLM guardrails beyond slot validation. The repo is public — github.com/[your-handle]/[repo-name]. Thank you."

B-roll: Cut to a single black slide with the GitHub URL. Hold for 5 seconds. Fade to black.

Three Sound-Bite Sentences to Memorize

1. **The hook:** "The only Track 3 submission that measures recovery, not guesses it."
2. **The AI justification:** "AI is in the classifier and the multilingual renderer. Every decision is deterministic Python. The AST test verifies this."
3. **The honest close:** "Here's what I do NOT claim to recover — and the exception list is on the dashboard, not buried in a footnote."

README Template (Copy-Paste Ready)

The README is the second-most-read artefact after the pitch video. The template below is a fill-in-the-blanks Markdown file that covers every section the judges will look for. Copy it into `README.md` at the repository root and fill in the bracketed placeholders.

```
[MARKDOWN]

# Hinglish Revenue Recovery Engine

> The only Track 3 submission with (a) real Razorpay test-mode closed-loop
> recovery, (b) P(recovery) measured — not heuristic — on a held-out batch
> with precision/recall and a retry-everything baseline, (c) decline-code-
> aware routing, and (d) multilingual customer comms — all under an
> AST-enforced policy gate with a replayable per-payment audit ledger.

## Problem

India processes 22.7 billion UPI transactions per month (Razorpay data,
June 2026). When a payment fails, recovering that revenue is a real
problem. Existing solutions fall short:

- **Stripe Smart Retries**: black-box ML on retry timing. Does not vary
channel based on failure reason. Does not coordinate with customer
communication. Does not support Indian languages.
- **Razorpay Agent Studio**: has Subscription Recovery and Abandoned Cart
Conversion agents, but they are English-only.
- **Purpose-built dunning tools** (Butterfly, Recurly): add ~20% recovery
on top of black-box retry, but are not India-specific.

**The gap**: decline-code-aware routing that varies the channel based on
the specific Razorpay error_code, coordinated with multilingual customer
communication, measured against baselines.

## Solution

A six-component pipeline (not an "agent"):
```

```
[continued]

1. **Webhook Ingestor** — receives `payment.failed` from Razorpay,
verifies HMAC-SHA256 over raw body, idempotent on `event_id`.
2. **Failure Diagnosis Engine** — classifies `error_code` into 5 buckets
(technical / funds / mandate / auth / fraud).
3. **P(Recovery) Classifier** — calibrated XGBoost + isotonic regression.
Outputs `p_recovery ∈ [0, 1]` with Brier score < 0.20 on holdout.
4. **Bounded Policy Gate** — five deterministic gates applied in order
(recoverable / velocity / DND / EVR / retry cap). AST-enforced:
no LLM in this component.
5. **Multilingual Communication Generator** — LLM fills slots inside
deterministic skeletons in 7 languages (en/hi/ta/te/kn/mr/bn).
6. **Recovery Executor + Audit Ledger** — creates Razorpay Payment Link,
listens for `payment_link.paid`, marks recovered, logs to hash-chained
SQLite + JSONL audit ledger.

## Architecture

```mermaid
flowchart LR
A[payment.failed webhook] --> B[Diagnosis Engine]
B --> C[P(Recovery) Classifier]
C --> D[Policy Gate 5 checks]
D --> E[Multilingual Comm Gen]
E --> F[Recovery Executor]
F --> G[payment_link.paid webhook]
G --> H[Audit Ledger]
```

See `docs/architecture.md` for full system design and `docs/decision_tree.md`
for the failure-decision tree.
```

```
[continued]


## Tech Stack

- **Backend**: Python 3.11, FastAPI, uvicorn
- **ML**: scikit-learn, XGBoost, scikit-learn CalibratedClassifierCV
- **Vector DB**: pgvector with HNSW (for customer_locale embeddings)
- **Embeddings**: multilingual-e5-base (7 languages)
- **LLM**: LiteLLM router (Claude / Gemini / OpenAI, configurable)
- **Storage**: SQLite (audit ledger) + JSONL (compliance stream)
- **Frontend**: Next.js 14 (dashboard)
- **Monitoring**: Evidently (drift), MLflow (model registry)
- **Tests**: pytest, pytest-asyncio, hypothesis (property-based)

## Quick Start

```bash
# 1. Clone and install
git clone https://github.com/[your-handle]/hinglish-revenue-recovery.git
cd hinglish-revenue-recovery
pip install -r requirements.txt

# 2. Configure Razorpay test mode
cp .env.example .env
# Edit .env: add RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, WEBHOOK_SECRET
# (Get these from https://dashboard.razorpay.com/app/keys)

# 3. Generate synthetic dataset
python scripts/generate_dataset.py --records 500 --out data/

# 4. Train classifier
```
```

```
[continued]

python scripts/train_model.py --data data/synthetic_failures.csv

# 5. Start the server
uvicorn app.main:app --reload --port 8000

# 6. (In another terminal) Expose webhook endpoint
ngrok http 8000
# Add the ngrok URL to Razorpay webhook settings
```

Razorpay Test Mode Setup

1. Create a Razorpay account at https://razorpay.com
2. Go to **Settings → API Keys → Generate Test Key**
3. Copy `Key ID` and `Key Secret` to `.env`
4. Go to **Settings → Webhooks → Add New Webhook**
5. URL: `https://[your-ngrok-url]/webhook/razorpay`
6. Events: `payment.failed`, `payment_link.paid`
7. Copy the **Webhook Secret** to `.env` as `WEBHOOK_SECRET`

See `docs/razorpay_setup.md` for screenshots.

Synthetic Dataset

The dataset is synthetic and honestly labelled as such. 500 records with
realistic distributions calibrated to Razorpay's published error_code
taxonomy. 70/30 train/holdout split.

```bash
python scripts/generate_dataset.py --records 500 --seed 42
```

```
[continued]

# Output: data/synthetic_failures.csv
```

See `data/schema.md` for the full schema and distribution rules.

API Reference

Endpoint	Method	Description
`/webhook/razorpay`	POST	Razorpay webhook receiver
`/api/recoveries`	GET	List recovery attempts
`/api/metrics`	GET	Get current metrics (money recovered, recovery rate, etc.)
`/api/audit/{payment_id}`	GET	Get full audit trail for a payment
`/api/dashboard`	GET	Dashboard HTML page

See `docs/api.md` for full OpenAPI spec.

Key Metrics (Holdout)

Metric	Value	Baseline (retry-everything)	Baseline (rule-based)
Money Recovered	₹4,82,150	₹1,83,400	₹2,99,800
Recovery Rate	62.4% (95% CI: 58.1% - 66.3%)	24.1%	39.2%
Precision	0.67	0.31	0.48
Recall	0.71	1.00	0.62
Brier Score	0.18	N/A	N/A
ECE	0.04	N/A	N/A

Bootstrap 95% CI from 2000 resamples of the 150-payment holdout.

```
[continued]

## Honest Exception List

We do NOT claim to recover:

- Mandate cancellations where the customer explicitly cancelled
- Confirmed-fraud blocks (`error_code: suspected_fraud`) — we route to
BLOCK, never retry
- Customer-initiated cancellations within 60s of payment attempt
- Payments > ₹50,000 (Razorpay test-mode cap)
- Unsupported customer locales (outside en/hi/ta/te/kn/mr/bn)
- Webhook delivery delays > 5 minutes (stale events)

## Failure Modes Handled

See `docs/failure_modes.md` for the full catalog of 8 failure modes with
detection, fallback, and audit logging for each. The TL;DR:

- LLM hallucinates a slot value → slot validation rejects, re-generates,
falls back to English template after 3 failures
- Razorpay API timeout → 502 to webhook (Razorpay retries), queue for retry
- Rate limit (429) → exponential backoff (1s/2s/4s/8s, max 5 retries)
- OOD error_code → manual_review queue, no auto-recovery
- Calibration drift → Evidently alert, fall back to per-code heuristic
- Concurrent retries → idempotency key conflict, reject duplicate

## What's NOT Included (Honest Limitations)

- **No production LLM guardrails** beyond slot validation (would add
Constitutional AI / Guardrails AI in production)
- **No real merchant pilot** — only synthetic data + Razorpay test mode
```

```
[continued]

- **No voice IVR integration** — voice scripts are generated but not
connected to a telephony provider (would use Exotel / Twilio in prod)
- **No multi-tenant auth** — single-tenant for the buildathon demo
- **Model trained on synthetic data only** — real production data would
change calibration; numbers reported here are methodology-validation,
not real-world performance claims

## Demo Video

5-minute pitch: [YouTube link / Loom link]

## Repository Structure

```
.
├── app/
| ├── main.py # FastAPI app
| ├── webhook.py # Razorpay webhook handler
| ├── diagnosis.py # Failure Diagnosis Engine
| ├── classifier.py # P(Recovery) Classifier (inference)
| ├── policy_gate.py # 5 deterministic gates
| ├── comm_generator.py # Multilingual message rendering (LLM-bounded)
| ├── recovery_executor.py # Razorpay Payment Link creation
| └── audit.py # Hash-chained audit ledger
├── rulebook/
| └── error_codes.yaml # Decline-code → channel mapping
├── scripts/
| ├── generate_dataset.py # Synthetic dataset generator
| ├── train_model.py # Classifier training + calibration
| └── compute_metrics.py # Bootstrap CI computation
```
```

```
[continued]

├── models/
│ └── p_recovery_calibrated.joblib
├── data/
│ └── synthetic_failures.csv
├── tests/
│ ├── test_invariants.py # AST-enforced: LLM bounded to comm_generator
│ ├── test_fault_injection.py # 8 failure modes triggered
│ └── test_policy_gate.py # 5 gates verified
├── docs/
│ ├── architecture.md
│ ├── failure_modes.md
│ ├── razorpay_setup.md
│ └── api.md
├── dashboard/ # Next.js 14 frontend
├── README.md
├── requirements.txt
└── .env.example
```

License

MIT License — see LICENSE

Acknowledgements

- Razorpay for the test-mode API and webhook infrastructure
- The [yvssshubam/razorpay-ai-buildathon](https://github.com/yvssshubam/razorpay-ai-buildathon) repo (Track 2) for the evaluation discipline pattern (holdout + bootstrap CI)
- The [kanikakataria75-ship-it/prahari-ai](https://github.com/kanikakataria75-ship-it/prahari-ai) repo (Track 2) for the AST-enforced invariants pattern
```

The README above is roughly 200 lines and covers every section judges will look for. The "Honest Exception List" and "What's NOT Included" sections are particularly important — they directly address the Measurement Honesty criterion by showing the builder knows where the system stops working, not just where it works. The acknowledgement of the two Track 2 leaders (yvssshubam, prahari-ai) shows the builder has done the competitive research and is bringing their patterns into Track 3.
```

Submission Checklist

The submission checklist is organised into three tiers: MUST-HAVE (the Round 2 requirements — missing any of these disqualifies), SHOULD-HAVE (the differentiators — missing these weakens the submission but does not disqualify), and NICE-TO-HAVE (the polish — these push the submission from "good" to "winning"). Each item has a verification method so the builder can check it programmatically rather than by feel.

Tier 1: MUST-HAVE (Disqualifying if Missing)

#	Item	Verification Method	Acceptance Criteria
1	Public GitHub repo	curl https://api.github.com/repos/{owner}/{repo}	HTTP 200, repo is public
2	README.md at root	ls README.md	File exists, > 100 lines
3	Architecture diagram	grep -r "mermaid|png|svg" docs/	At least 1 architecture diagram present
4	5-min pitch video	YouTube/Loom URL accessible	Video is 4:30-5:30 long, public
5	Working code	python -m pytest tests/	Tests pass, no ImportError
6	Measured metrics	cat docs/metrics.md	Holdout AUC + precision + recall reported
7	.env.example	ls .env.example	File exists, contains RAZORPAY_KEY_ID etc.
8	requirements.txt	pip install -r requirements.txt	Installs without errors

Tier 2: SHOULD-HAVE (Differentiators)

#	Item	Verification Method	Acceptance Criteria
9	Failure modes table	cat docs/failure_modes.md	⩾ 8 failure modes with detection + fallback
10	Honest exception list	grep -A 20 "Exception" README.md	⩾ 5 explicit "NOT claimed" items
11	Baseline comparison	cat docs/metrics.md	⩾ 2 baselines computed on same holdout
12	Audit trail sample	cat docs/audit_sample.jsonl	⩾ 1 full decision → attempt → outcome trace
13	Razorpay test-mode integration	grep -r "razorpay.Client" app/	Real SDK calls, not mocked

#	Item	Verification Method	Acceptance Criteria
14	AST-invariant test	pytest tests/test_invariants.py -v	Passes; build fails if LLM in policy
15	Multilingual showcase	ls docs/hinglish_samples/	⩾ 4 exhibit scripts in ⩾ 3 languages
16	Cost-sweep analysis	cat docs/cost_sweep.md	Reports regime where model loses

Tier 3: NICE-TO-HAVE (Polish)

#	Item	Verification Method	Acceptance Criteria
17	Live demo in pitch video	Watch video segment 4	90s of live webhook flow
18	Dashboard screenshot	ls dashboard/screenshot.png	PNG present, shows hero KPIs
19	Bootstrap 95% CI	cat docs/metrics.md	CI reported for ⩾ 3 metrics
20	Hash-chained audit	python scripts/verify_chain.py	Chain verification passes
21	Calibration plot	ls docs/calibration.png	Reliability diagram present
22	Cost-weighted confusion matrix	cat docs/metrics.md	☒ cost of false positive reported

Final 60-Minute Pre-Deadline Checklist

In the final hour before the deadline, run through this checklist in order. Each item takes roughly 5 minutes. If any item fails, fix it before moving to the next — do not skip and come back, because the next item may depend on the previous one.

1. **Repo is public** — verify with `git remote -v` and check GitHub URL in incognito mode
2. **README renders correctly** — view on GitHub, check that all sections are populated (not [TBD])
3. **Video is uploaded and public** — verify URL opens in incognito, check duration is 4:30-5:30
4. **Video URL is in README** — search README for the video link
5. **.env.example has all required keys** — diff against `app/config.py`
6. **Tests pass on a clean clone** — fresh `git clone` + `pip install` + `pytest`
7. **Architecture diagram is readable** — view at 100% zoom, all text legible
8. **Metrics file has actual numbers (not [TBD])** — verify holdout AUC, precision, recall, Brier are populated

9. **Exception list is in README** — search for "NOT claim" or "Exception"
10. **Submission form is filled** — repo URL, video URL, track selection, all required fields

Risk/Decision Matrix — When to Pivot

The risk/decision matrix is consulted at the end of each kill-gate day. It is a pre-committed decision tree: at each gate, the builder checks the signal, finds the row in the matrix, and executes the corresponding action. The point of pre-committing is to remove decision fatigue at 11pm on Day 4 — the matrix says what to do, the builder does it.

The matrix below is more detailed than the simplified version in Section 13. It includes the signal (what to measure), the threshold (what value triggers each action), the action (continue / simplify / pivot), and the cost of each path.

Day 2 Kill Gate — Classifier AUC

AUC on Holdout	Action	What This Means
> 0.75	Continue with full scope	Model is strong; all 4 pillars viable
0.65 - 0.75	Continue, simplify features	Drop rare error_codes from training; merge into "other" bucket
0.55 - 0.65	Continue, drop AST-invariant claim	Model is weak; rely on rule-based baseline as primary
< 0.55	PIVOT TO TRACK 2	Model not better than random; Track 3 thesis (measured P) fails

Day 2 is the most important kill gate because the entire Track 3 thesis depends on having a model that predicts recovery better than random. If AUC is below 0.55, the model is not adding information, and the "measured P(recovery)" pillar collapses. The fallback is Track 2 (Risk Manager), where the rule-based per-code lookup is acceptable because Track 2's bar is measured chargeback triage — the participant's existing fraud detection code can be repurposed.

Day 3 Kill Gate — Webhook Closed Loop

Signal	Action	What This Means
`HMAC verify + idempotency + payment_link.paid handler all work`	Continue with real Razorpay integration	Pillar 1 (closed loop) is real
`HMAC works but payment_link.paid not received`	Continue with mocked payment_link.paid	Trigger payment_link.paid manually in tests; label as "partial mock" in README
HMAC verification fails	Debug signature computation (raw body vs parsed)	Most common bug: verifying signature over JSON instead of raw bytes

Signal	Action	What This Means
Webhook never arrives from Razorpay	Use ngrok tunnel, verify webhook URL in Razorpay dashboard	Most common bug: webhook URL not registered
Cannot get any webhook to fire	Mock the entire webhook layer	Label as "mocked webhook" in README + pitch; do NOT fake the recovery outcome

Day 3 is the second-most important kill gate because the "real Razorpay integration" pillar depends on the closed loop working. If it cannot be made to work, the honest fallback is to mock the webhook layer and label it as such in the README and pitch video. This is what differentiates the submission from PRANEETHWARANK, which fakes the Razorpay integration with a stub that returns the ground-truth label as the "recovery outcome" — that is dishonest, and judges will catch it.

Day 4 Kill Gate — Policy Gate + AST Invariant

Signal	Action	What This Means
All 5 gates pass fault-injection + AST test passes	Continue with full bounded-explainability claim	Pillar 4 (AST gate) is real
5 gates pass but AST test fails to add	Ship 5 gates without AST test	Weaken claim to "5 deterministic gates" without machine verification
Only 3 of 5 gates pass fault-injection	Drop the failing 2 gates	Common drop: velocity + DND gates (keep recoverable + EVR + retry cap)
AST test passes but no LLM in codebase to test against	Add a stub LLM call in policy_gate, verify test catches it	Validates the AST test before the real LLM is added

Day 5 Kill Gate — Multilingual Comms

Signal	Action	What This Means
All 7 languages render valid messages	Continue with 7 languages	Pillar 3 (multilingual) fully realised
4-6 languages work, others fail slot validation	Drop to 4 languages (en/hi/ta/te)	Still beats competitors (zero have any)
Only en + hi work	Drop to 2 languages	Minimum viable differentiation — still better than competitors
No language renders valid messages	Drop multilingual entirely, rely on English templates	Pillar 3 lost; rely on Pillars 1, 2, 4 alone

Day 6 Kill Gate — Dashboard + Metrics

Signal	Action	What This Means
Dashboard reads live metrics.json + bootstrap CI computed	Continue with full dashboard	Pillar 2 (measured) fully realised
Dashboard works but no bootstrap CI	Compute bootstrap CI offline, hardcode in dashboard	Weaker but still measured
Dashboard broken but metrics.json exists	Screenshot of metrics.json + manual chart in README	Static evidence, not live dashboard
No metrics computed at all	Use Day 7 buffer to compute at least 3 metrics	Critical — cannot submit without metrics

Track 2 Fallback Plan

If Day 2 kill gate triggers a pivot (< 0.55 AUC), the fallback is Track 2 (AI Risk Manager). The participant's existing fraud detection repo + calibration/Fairlearn work maps directly. The fallback plan is:

1. **Day 3 (pivot day)**: re-scope the existing fraud detection repo for chargeback triage. Reuse the calibration code. Frame as "return-risk scorer with cost-weighted metrics".
2. **Day 4-5**: implement evidence assembly (auto-gather order history, delivery proof, device/IP match) and cost-weighted confusion matrix (☒ cost of false positive = lost legitimate sale).
3. **Day 6-7**: holdout evaluation with two baselines (contest-everything / contest-nothing) and bootstrap CI, à la yvssshubam.
4. **Day 8-9**: README + pitch video + submission. Track 2 is lower-risk but lower-ceiling; the pitch focuses on "measured chargeback triage with honest cost-weighted metrics".

> **KILL CRITERIA SUMMARY**
> The four kill gates are: Day 2 AUC < 0.55 (pivot to Track 2), Day 3 webhook loop broken (mock honestly), Day 5 multilingual fails (drop to fewer languages), Day 6 no metrics (use Day 7 buffer). Each gate has a pre-committed action so the builder does not need to decide at 11pm. The matrix is the safety net — it ensures the submission is always shippable, even if every gate fails.

Closing — Why This Wins

The submission wins because it occupies a cell in the competitive landscape that no other submission currently occupies. The four pillars — real Razorpay closed-loop, measured P(recovery), decline-code-aware routing, and multilingual comms under an AST-enforced policy gate — are each independently defensible, and together they form a position that no competitor has. The competitor research shows that the field is full of submissions with one or two of these properties, but none with all four.

The strategic move is not to build the most ambitious submission. It is to build the most honest one. The judging criteria reward measured honesty over ambition: "a 100% working solution with modest metrics beats a 70% working ambitious build". The submission's design — explicit baselines, bootstrap confidence intervals, an honest exception list on the dashboard, and a published regime where the model loses (à la prahari-ai) — is calibrated to this criterion.

Four Pillars Recap

1. **Real Razorpay test-mode closed-loop recovery**: payment.failed → link → payment_link.paid. Only madhav9757 has this among Track 3 competitors, and they route on heuristics.
2. **P(recovery) measured on a held-out batch**: calibrated classifier with Brier score, ECE, precision/recall, two baselines, bootstrap 95% CI. Nobody in Track 3 has this.
3. **Decline-code-aware routing**: Razorpay error_code → channel mapping, hand-coded and reviewable. Beats Stripe Smart Retries' black-box retry timing.
4. **Multilingual customer comms under AST-enforced policy gate**: 7 languages, LLM bounded to slot-filling inside deterministic skeletons, AST test verifies LLM never decides.

What to Do Today (Next 24 Hours)

If you are reading this document before Day 1, the three things to do in the next 24 hours are:

1. **Register a Razorpay test-mode account** at https://razorpay.com. Generate API keys. Add a webhook endpoint (use ngrok for local development). Trigger a test payment failure and verify the webhook fires. This de-risks Day 3 (the highest-risk kill gate).
2. **Generate the 500-record synthetic dataset** using the schema and distribution rules in Section 5. The dataset generator script can be written in 2 hours. Once it exists, you can start Day 2 (classifier training) immediately.
3. **Write the failure-decision tree skeleton** in `policy_gate.py`. Just the function signatures and the gate order — no implementation yet. This forces you to think through the gate logic before Day 4, when the implementation pressure is highest.

One-Sentence Elevator Pitch (Memorize)

> **MEMORIZE THIS SENTENCE**
> The only Track 3 submission with real Razorpay test-mode closed-loop recovery, measured P(recovery) on a held-out batch with baselines and bootstrap CI, decline-code-aware routing, and multilingual customer comms — under an AST-enforced policy gate with a replayable audit ledger.

This is the sentence you say at the start of the pitch video, in the README's tagline, and in any conversation with judges. It is the positioning sentence — every word matters, every claim is defensible, and together they form the empty cell in the competitive landscape that this submission is designed to occupy.

The next nine days are execution. The strategy is set. Build honestly, measure rigorously, and document the failures with the same care as the successes. That is how this wins.